

MANUAL COMPLETO DE ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORES I

Grado en Ingeniería Informática - 1º Curso

Este documento unifica todos los temas del plan de estudio de arquitectura interna, formato de instrucciones, programación ARM de bajo nivel, gestión de pila y periféricos de E/S en un único manual para facilitar su lectura, impresión o conversión a formatos como PDF.

Índice General del Manual

- **Bloque 1: La Arquitectura de Máquina y el Repertorio (Semanas 1 a 5)**
 - Tema 1: Conceptos Fundamentales: Lenguaje Máquina vs. Ensamblador ARM
 - Tema 2: Banco de Registros y Registro de Estado (CPSR) en ARM
 - Tema 3: Formato y Codificación de Instrucciones Máquina en ARM
 - Tema 4: Modos de Direccionamiento en ARM
 - Tema 5: Procesamiento de Datos y Control de Secuenciamiento en ARM
- **Bloque 2: Programación Estructurada y Gestión de Memoria (Semanas 6 a 11)**
 - Tema 6: Traducción de Estructuras de Control a Ensamblador ARM
 - Tema 7: El Mecanismo de Llamada a Subrutinas y el Link Register (LR)
 - Tema 8: La Pila (Stack) y la Preservación del Contexto en ARM
 - Tema 9: Bloques de Activación (Stack Frames) y Gestión de Variables Locales
- **Bloque 3: Interoperabilidad y Periféricos (Semanas 12 a 15)**
 - Tema 10: Interoperabilidad de Lenguajes: Código C y Ensamblador ARM
 - Tema 11: El Subsistema de Entrada/Salida: Registros del Controlador
 - Tema 12: Métodos de Transferencia: Polling o Consulta de Estado
 - Tema 13: Mecanismo de Interrupciones y Excepciones en ARM
- **Secciones Finales**
 - Glosario de Términos
 - Bibliografía Recomendada

Tema 1: Conceptos Fundamentales: Lenguaje Máquina vs. Ensamblador ARM

Para comprender la ejecución de un programa en un computador, es imprescindible estudiar los niveles de abstracción más cercanos al hardware físico. La CPU no entiende lenguajes de programación de alto nivel como Java, C++ o Python; procesa únicamente secuencias binarias de unos y ceros en la memoria.

1. Niveles de Abstracción: Alto Nivel, Ensamblador y Lenguaje Máquina

- **Lenguaje de Alto Nivel:** Diseñado para ser legible por humanos. Abstrae la memoria física mediante variables, objetos y estructuras de control dinámicas.
- **Lenguaje Ensamblador (Assembly):** Representación de bajo nivel basada en nemotécnicos (abreviaturas como `ADD`, `SUB`, `MOV`) que tienen una correspondencia directa (generalmente 1 a 1) con las instrucciones de la máquina. Depende estrictamente de la arquitectura del procesador (ARM, x86, MIPS).
- **Lenguaje Máquina:** Código binario ejecutable interpretado directamente por la Unidad de Control del procesador. Se representa frecuentemente en hexadecimal para facilitar su lectura en herramientas de depuración.

El Proceso de Construcción del Software (Pipeline de Compilación)

1. **Compilador:** Traduce el código fuente en alto nivel (ej. `programa.c`) a código ensamblador (`programa.s`).
 2. **Ensamblador:** Convierte el código ensamblador en un fichero objeto binario (`programa.o`) que contiene instrucciones máquina no enlazadas y tablas de símbolos.
 3. **Enlazador (Linker):** Combina múltiples ficheros objeto y bibliotecas de sistema, resolviendo las direcciones de memoria de variables y funciones globales, para generar el fichero ejecutable binario final (ej. `programa.elf` o `programa.exe`).
-

2. Filosofía RISC y la Arquitectura ARM

ARM (Advanced RISC Machines) es la arquitectura de procesadores más difundida en el mundo de los dispositivos móviles, consolas portátiles y sistemas empuetrados. Su diseño se rige bajo los principios **RISC (Reduced Instruction Set Computer)**:

- **Instrucciones de longitud fija:** Todas las instrucciones de ARM de 32 bits ocupan exactamente 32 bits (4 bytes) de memoria, lo que simplifica la decodificación por hardware de la Unidad de Control.
 - **Arquitectura Load/Store (Carga/Almacenamiento):** El procesador no puede realizar operaciones lógicas o aritméticas directamente sobre posiciones de la memoria RAM. Los datos deben ser cargados en los registros del procesador (**LDR**), procesados en el banco de registros (ej. **ADD**) y luego almacenados de vuelta en la memoria (**STR**).
 - **Repertorio de instrucciones ortogonal:** Instrucciones simples que se ejecutan habitualmente en un solo ciclo de reloj.
 - **Banco de registros amplio:** En lugar de usar acumuladores o variables de memoria intermedias, RISC dispone de múltiples registros de propósito general veloces dentro de la propia CPU.
-

3. Entornos de Simulación y Laboratorio

El aprendizaje práctico de la programación en ensamblador ARM se facilita mediante simuladores que permiten observar la CPU por dentro de forma interactiva:

- **VisUAL (Visual ARM Simulator):** Un simulador de código abierto especialmente diseñado para estudiantes de ingeniería. Permite ejecutar programas de procesamiento de datos en ARMv7, ver la evolución temporal del banco de registros, los bits del registro de estado CPSR y simular memoria RAM direccionable.
-

4. El Toque Informático

Simulación de un Micro-Ensamblador/Desensamblador

Como ingenieros informáticos, podemos programar herramientas que faciliten la traducción de bajo nivel. El siguiente script de Python actúa como un desensamblador rudimentario de instrucciones máquina binarias simuladas para ARM, abstrayendo cómo la CPU analiza los bits del opcode:

```

def desensamblar_instruccion(codigo_hex):
    # Convertir código hexadecimal a binario de 32 bits relleno con ceros a la izquierda
    inst_bin = bin(int(codigo_hex, 16))[2:].zfill(32)

    # Simular la decodificación de campos de una instrucción de procesamiento de datos
    ARM
    condicion = inst_bin[0:4]
    opcode = inst_bin[4:8]
    rd = int(inst_bin[8:12], 2)
    rn = int(inst_bin[12:16], 2)
    operando2 = int(inst_bin[16:32], 2)

    # Mapeo básico de opcodes
    opcodes = {
        "0100": "ADD",
        "0010": "SUB",
        "1101": "MOV",
        "0000": "AND"
    }

    # Mapeo de condiciones
    condiciones = {
        "1110": "AL (Siempre)",
        "0000": "EQ (Igual)",
        "0001": "NE (No igual)"
    }

    op_name = opcodes.get(opcode, "UNKNOWN")
    cond_name = condiciones.get(condicion, "UNKNOWN")

    print(f"Código Hex: 0x{codigo_hex}")
    print(f" - Binario: {inst_bin}")
    print(f" - Condición: {condicion} ({cond_name})")
    print(f" - Opcode: {opcode} ({op_name})")
    print(f" - Registro D: R{rd}")
    print(f" - Registro N: R{rn}")
    print(f" - Operando 2: {operando2} (en decimal)")
    print(f"Traducción ASM sugerida: {op_name} R{rd}, R{rn}, #{operando2}")
    print("-" * 50)

    # Simulación: Decodificar una instrucción ADD R1, R2, #45
    # Representada con condición AL (1110), opcode ADD (0100), Rd (R1: 0001), Rn (R2: 0010),
    # Operand2 (45: 0000000000101101)
    # 1110 0100 0001 0010 0000000000101101 en binario -> E412002D en Hexadecimal
    desensamblar_instruccion("E412002D")

```

5. Ejercicios Resueltos

Ejercicio 1

Diferenciar entre un compilador, un ensamblador y un enlazador en el ciclo de vida del desarrollo de software.

Solución:

- **Compilador:** Recibe un archivo escrito en lenguaje de alto nivel (como C) y lo traduce a código ensamblador (legible para el programador de bajo nivel, con nemotécnicos de la arquitectura).
- **Ensamblador:** Recibe el archivo en ensamblador y traduce sus instrucciones a binario puro (código máquina), generando un archivo objeto (.o).
- **Enlazador:** Une los distintos archivos objeto del proyecto y resuelve las direcciones absolutas de las variables y funciones externas y librerías, empaquetando todo en un único fichero ejecutable por el sistema operativo.

Ejercicio 2

Explicar por qué en una arquitectura RISC como ARM no es posible realizar una suma directa de una variable de memoria RAM con el contenido de un registro del procesador.

Solución: ARM sigue estrictamente la arquitectura **Load/Store**. El bus de datos y la ALU (Unidad Aritmético-Lógica) están integrados dentro de la CPU para operar únicamente con operandos situados en los registros internos. La memoria RAM es un subsistema externo lento. Para realizar la operación, se debe seguir la siguiente secuencia obligatoria:

1. Cargar el dato de la memoria RAM a un registro temporal (ej. `LDR R0, [R1]`).
2. Realizar la suma sobre el registro (ej. `ADD R0, R0, R2`).
3. Almacenar el resultado de vuelta en la memoria RAM (ej. `STR R0, [R1]`).

6. Ejercicios Propuestos

1. Dada una instrucción con el código hexadecimal `0xE021000A` en el simulador, explica el significado de codificar la información en hexadecimal en lugar de binario directo para el programador.
2. Investiga el papel de los nemotécnicos en ensamblador y determina si existe una traducción siempre directa de 1 a 1 entre una línea de código ensamblador y una instrucción máquina de 32 bits de ARM.
3. Describe brevemente las diferencias operativas entre la arquitectura x86 (típica de ordenadores personales) y ARM (RISC). ¿Por qué ARM consume significativamente menos energía que x86?

Tema 2: Banco de Registros y Registro de Estado (CPSR) en ARM

El diseño interno de un procesador se expone al programador a través de su **Modelo de Programación**, constituido por el banco de registros internos y el registro de estado. En la arquitectura ARM de 32 bits, el programador tiene acceso directo a 16 registros principales y a un registro especial de control.

1. El Banco de Registros de ARM (Modo Usuario)

En el modo de ejecución normal (Modo Usuario), el procesador ARM dispone de **16 registros visibles** de 32 bits, denominados de **R0** a **R15**, más el registro de estado **CPSR**.

Registro	Alias	Función Principal
R0 - R3	-	Registros de propósito general. Se usan para pasar parámetros a funciones y retornar valores.
R4 - R10	-	Registros de propósito general. Conservan su valor tras llamadas a subrutinas (AAPCS).
R11	FP	<i>Frame Pointer</i> (Puntero de Marco). Opcional, marca el inicio del bloque de activación en la pila.
R12	IP	<i>Intra-Procedure-call scratch register</i> . Registro temporal usado por el enlazador dinámico.
R13	SP	<i>Stack Pointer</i> (Puntero de Pila). Almacena la dirección de memoria del elemento en la cima de la pila.
R14	LR	<i>Link Register</i> (Registro de Enlace). Guarda la dirección de retorno al llamar a una subrutina.
R15	PC	<i>Program Counter</i> (Contador de Programa). Apunta a la dirección de la instrucción en ejecución (con pipeline, apunta a $PC + 8$).

2. El Registro de Estado CPSR (Current Program Status Register)

El **CPSR** es un registro dedicado de 32 bits que contiene información sobre el estado actual del procesador. Se divide en campos de flags y de control:

Bit	31	30	29	28	...	7	6	5	4	3	2	1	0														
	[	N		Z		C		V	]	...	[	I		F		T		M4		M3		M2		M1		M0	]
	_____Flags_____/												_____Control_____/														

Los Flags de Condición (Bits 28 a 31)

Son modificados de forma automática por instrucciones aritméticas/lógicas específicas (si añadimos el sufijo **S**, ej: **ADDS**, **SUBS**) o por instrucciones de comparación (**CMP**, **TST**).

- **N (Negative - Bit 31)**: Se activa ($N = 1$) si el resultado de la última operación fue negativo (el bit 31 del resultado es 1).
- **Z (Zero - Bit 30)**: Se activa ($Z = 1$) si el resultado de la última operación fue exactamente cero.
- **C (Carry - Bit 29)**: Se activa ($C = 1$) si hubo acarreo en una suma sin signo (el resultado supera $2^{32} - 1$) o si no hubo necesidad de préstamo (*borrow*) en una resta (es decir, el minuendo fue mayor o igual que el sustraendo).
- **V (oVerflow - Bit 28)**: Se activa ($V = 1$) si se produce un desbordamiento aritmético al operar con números con signo (ej. sumar dos números positivos y obtener un resultado negativo).

Bits de Control (Bits 0 a 7)

- **I y F (Interrupt Masks)**: Deshabilitan las interrupciones externas normales (IRQ) e interrupciones rápidas (FIQ) cuando valen 1.
- **T (State bit)**: Define si el procesador está ejecutando en modo ARM (32 bits, $T = 0$) o modo Thumb (16 bits, $T = 1$).
- **M[4:0] (Mode bits)**: Indican el modo de operación actual del procesador (Usuario, System, FIQ, IRQ, Supervisor, Abort, Undefined). El cambio de modo permite el aislamiento de recursos entre el software de usuario y el núcleo del sistema operativo.

3. El Toque Informático

Simulador del Estado de Registros y Flags

Para comprender cómo las instrucciones máquina modifican los flags de la CPU, implementamos este simulador interactivo en Python que modela la ALU del procesador ARM actualizando el banco de registros y el CPSR:

```
class ARMCPU:
    def __init__(self):
        # 16 registros de 32 bits (R0-R15)
        self.registers = [0] * 16
        # Flags CPSR: N, Z, C, V (inicializados en 0)
        self.flags = {"N": 0, "Z": 0, "C": 0, "V": 0}

    def print_state(self):
        print("=== BANCO DE REGISTROS ===")
        for r in range(0, 16, 4):
            print(f"R{r:02d}: 0x{self.registers[r]:08X} | R{r+1:02d}: "
                  f"0x{self.registers[r+1]:08X} | "
                  f"R{r+2:02d}: 0x{self.registers[r+2]:08X} | R{r+3:02d}: "
                  f"0x{self.registers[r+3]:08X}")
            print("FLAGS CPSR:")
            print(f"  N (Negativo): {self.flags['N']} | Z (Cero): {self.flags['Z']} | "
                  f"C (Acarreo): {self.flags['C']} | V (Desbordamiento): "
                  f"{self.flags['V']}")
            print("=" * 55)

    def ejecutar_subs(self, rd, rn, rm_val):
        # Operación: Rd = Rn - Rm_val
        rn_val = self.registers[rn]
        resultado = (rn_val - rm_val) & 0xFFFFFFFF
        self.registers[rd] = resultado

        # Actualizar flags condicionales (simulación de SUBS)
        # 1. Flag Z
        self.flags["Z"] = 1 if resultado == 0 else 0
        # 2. Flag N
        self.flags["N"] = 1 if (resultado & 0x80000000) != 0 else 0
        # 3. Flag C (En restas en ARM, C=1 si no hay préstamo, es decir, Rn >= Rm)
        self.flags["C"] = 1 if rn_val >= rm_val else 0
        # 4. Flag V (Desbordamiento de signo)
        # Ocurre si restamos signos opuestos y el resultado tiene signo incorrecto
        signo_rn = (rn_val & 0x80000000) != 0
        signo_rm = (rm_val & 0x80000000) != 0
        signo_res = (resultado & 0x80000000) != 0
        if signo_rn != signo_rm and signo_res != signo_rn:
            self.flags["V"] = 1
        else:
            self.flags["V"] = 0

# Inicialización y simulación de la instrucción: SUBS R0, R1, R2
cpu = ARMCPU()
```

```
cpu.registers[1] = 10 # R1 = 10
cpu.registers[2] = 10 # R2 = 10
print("Antes de ejecutar SUBS R0, R1, R2:")
cpu.print_state()

cpu.ejecutar_subs(rd=0, rn=1, rm_val=cpu.registers[2])
print("Después de ejecutar SUBS R0, R1, R2 (resultado 0):")
cpu.print_state()
```

4. Ejercicios Resueltos

Ejercicio 1

Indicar la función específica del registro **R14 (LR)** y explicar qué ocurre cuando una función secundaria llama a una tercera (llamadas anidadas) sin salvar este registro.

Solución: El **R14** o **LR** (Link Register) almacena de forma automática la dirección de memoria a la que la CPU debe retornar tras terminar de ejecutar la subrutina actual. Al ejecutar la instrucción **BL subrutina**, el hardware guarda la dirección de la siguiente instrucción en **LR**.

Si hay **llamadas anidadas**:

1. La función **Principal** ejecuta **BL FuncionA**. El hardware guarda la dirección de retorno a **Principal** en **LR**.
2. **FuncionA** ejecuta **BL FuncionB**. El hardware sobrescribe **LR** con la dirección de retorno a **FuncionA**.
3. Al finalizar **FuncionB**, retorna haciendo **MOV PC, LR** (volviendo con éxito a **FuncionA**).
4. Cuando **FuncionA** termina e intenta retornar haciendo **MOV PC, LR**, se produce un **bucle infinito** porque **LR** todavía contiene la dirección de retorno a **FuncionA**, no a **Principal**. Para evitar esto, **FuncionA** debe guardar **LR** en la pila (Stack) al inicio y recuperarlo al final.

Ejercicio 2

Si ejecutamos una resta que da como resultado un número negativo, ¿cómo cambian las banderas **N** y **Z** en el registro de estado **CPSR**?

Solución:

- **Z (Zero):** Se pondrá en 0, ya que el resultado es diferente de cero.
 - **N (Negative):** Se pondrá en 1, porque el bit más significativo (bit 31) del resultado en binario de 32 bits estará a 1, indicando que representa un valor negativo bajo complemento a 2.
-

5. Ejercicios Propuestos

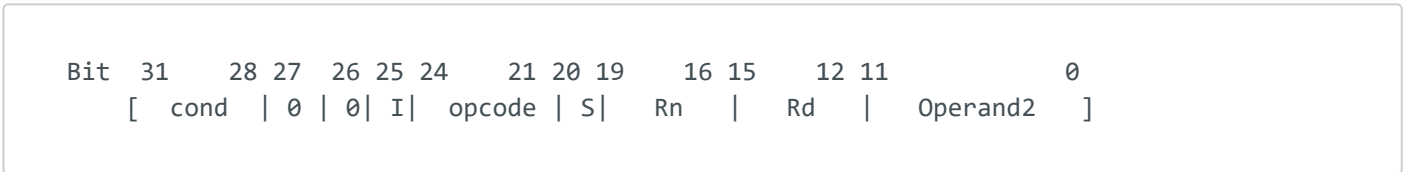
1. Dada la instrucción `CMP R0, R1`, explica detalladamente qué operación matemática realiza la CPU internamente y si modifica el registro de destino `R0`.
2. Analiza la diferencia en la modificación de banderas al ejecutar `ADD R0, R1, R2` frente a `ADDS R0, R1, R2`. ¿Por qué por defecto ARM no actualiza las banderas del CPSR en todas sus instrucciones de datos?
3. Investiga el papel de los modos de ejecución del procesador ARM. ¿Por qué el código del sistema operativo se ejecuta en modo Supervisor (SVC) y el código de las aplicaciones en modo Usuario (USR)?

Tema 3: Formato y Codificación de Instrucciones Máquina en ARM

Una de las características clave de la arquitectura RISC de ARM de 32 bits es la regularidad en el formato de sus instrucciones. Para permitir una decodificación veloz mediante hardware, todas las instrucciones se codifican en palabras de longitud fija de exactamente **32 bits** (4 bytes).

1. El Formato General de una Instrucción ARM

La Unidad de Control del procesador interpreta los bits de una instrucción dividiéndolos en campos específicos. El formato básico para las instrucciones de procesamiento de datos es:



Descripción Detallada de los Campos

- cond (Bits 31-28):** Campo de Condición. Permite la ejecución condicional de casi cualquier instrucción en ARM.
 - 1110 (0xE): Ejecución Incondicional (**AL** - *Always*).
 - 0000 (0x0): Ejecución si es Igual (**EQ** - *Equal*).
 - 0001 (0x1): Ejecución si es Diferente (**NE** - *Not Equal*).
- Tipo de Instrucción (Bits 27-26):** Para procesamiento de datos, estos bits siempre valen 00.
- I - Immediate flag (Bit 25):**
 - I = 0: El campo **Operand2** es un registro (y opcionalmente puede ser procesado por el Barrel Shifter).
 - I = 1: El campo **Operand2** representa un valor inmediato (constante numérica).
- opcode (Bits 24-21):** Define la operación específica.
 - 0100 (0x4): **ADD** (Suma)
 - 0010 (0x2): **SUB** (Resta)

- **1101** (0xD): **MOV** (Mover/Copiar)
- **0000** (0x0): **AND** (Y lógica)

5. **S (Bit 20): Set Condition Codes.**

- **S = 0**: No modifica las banderas del registro CPSR.
- **S = 1**: Modifica las banderas (N, Z, C, V) en función del resultado.

6. **Rn (Bits 19-16):** Registro del primer operando fuente (4 bits permiten codificar de R0 a R15).

7. **Rd (Bits 15-12):** Registro de destino. Almacena el resultado de la operación.

8. **Operand2 (Bits 11-0):** Operando secundario. Ocupa 12 bits y su interpretación depende del bit **I**.

2. La Codificación de Valores Inmediatos (Operand2 con I=1)

El campo **Operand2** tiene solo 12 bits libres. Esto imposibilita representar cualquier constante de 32 bits de forma directa. ARM solventa esto dividiendo los 12 bits del operando inmediato en dos subcampos:

- **Rotación (Bits 11-8):** 4 bits para codificar un valor de rotación par (multiplicado por 2), dando un rango de rotación de 0 a 30 posiciones a la derecha.
- **Valor Base (Bits 7-0):** 8 bits para almacenar una constante entre 0 y 255.

La fórmula de decodificación de la constante final es:

$$\text{Constante} = \text{Valor Base rotado a la derecha } (2 \cdot \text{Rotación}) \text{ bits}$$

Esto permite representar constantes útiles como potencias de dos, máscaras de bits comunes o múltiplos de página de forma eficiente en solo 12 bits de espacio de instrucción física.

3. El Toque Informático

Compilador/Codificador de Instrucciones Ensamblador ARM

El siguiente script en Python simula el trabajo del ensamblador convirtiendo una cadena de texto de código ensamblador ARM simple a su representación en código de máquina de 32 bits (tanto en binario como en hexadecimal):

```
def ensamblar_datos(inst_asm):
    # Ejemplo simple para parsear: "ADD R1, R2, #45" o "SUB R3, R4, #12"
    partes = inst_asm.replace(" ", "").split()

    cond = "1110" # AL (Siempre)
    tipo = "00"
    i_flag = "1" # Usaremos inmediato para simplificar

    opcodes = {"ADD": "0100", "SUB": "0010", "MOV": "1101"}
    opcode = opcodes.get(partes[0], "0000")

    s_flag = "1" if partes[0].endswith("S") else "0"
    # Quitar la S para mapear el opcode básico
    basic_op = partes[0].rstrip("S")
    opcode = opcodes.get(basic_op, "0000")

    rd = bin(int(partes[1].replace("R", "")))[2:].zfill(4)
    rn = bin(int(partes[2].replace("R", "")))[2:].zfill(4)

    # Inmediato de 12 bits (asumiendo rotación 0 para valores < 256)
    val_inmediato = int(partes[3].replace("#", ""))
    operand2 = bin(val_inmediato)[2:].zfill(12)

    inst_binaria = cond + tipo + i_flag + opcode + s_flag + rn + rd + operand2
    inst_hex = f"{int(inst_binaria, 2):08X}"

    print(f"Línea Ensamblador: {inst_asm}")
    print(f"Campos Binarios: [{cond}] [{tipo}] [{i_flag}] [{opcode}] [{s_flag}] [{rn}] [{rd}] [{operand2}]")
    print(f"Instrucción Binaria: {inst_binaria}")
    print(f"Código de Máquina: 0x{inst_hex}")
    print("-" * 65)

# Probar codificación
ensamblar_datos("ADD R1, R2, #45")
ensamblar_datos("SUB R3, R4, #12")
```

4. Ejercicios Resueltos

Ejercicio 1

Dada la instrucción máquina en hexadecimal **0xE2831005**, decodificar todos sus campos e identificar la instrucción ensamblador equivalente.

Solución:

1. **Convertir Hexadecimal a Binario de 32 bits:** $0xE2831005 \Rightarrow 1110\ 0010\ 1000\ 0011\ 0001\ 0000\ 0000\ 0101$
2. **Segmentar los campos:**
 - **cond** (bits 31-28): $1110 \Rightarrow AL$ (Siempre/Incondicional).
 - **tipo** (bits 27-26): $00 \Rightarrow$ Procesamiento de datos.
 - **I** (bit 25): $1 \Rightarrow$ Operando inmediato.
 - **opcode** (bits 24-21): $0100 \Rightarrow ADD$.
 - **S** (bit 20): $0 \Rightarrow$ No actualiza flags.
 - **Rn** (bits 19-16): $0011 \Rightarrow$ Registro $R3$.
 - **Rd** (bits 15-12): $0001 \Rightarrow$ Registro $R1$.
 - **Operand2** (bits 11-0): $0000\ 0000\ 0101 \Rightarrow$ Inmediato base 5 con rotación 0.
3. **Ensamblar la instrucción final:** $ADD\ R1,\ R3,\ \#5$

Ejercicio 2

Explicar por qué una instrucción como $ADD\ R0,\ R1,\ \#300$ no se puede codificar con rotación cero en el operando inmediato y cómo lo resuelve el ensamblador.

Solución: El campo de valor base para el inmediato en **Operand2** tiene un límite de 8 bits (números del 0 al 255). Por lo tanto, el número 300 es demasiado grande para caber de forma directa en el byte de base. Para solventar esto, el ensamblador realiza una búsqueda de una rotación par adecuada:

- 300 en binario es $00000001\ 00101100$.
- Podemos representarlo como el valor base 75 (binario 01001011) rotado a la derecha por 30 bits (que equivale a una rotación de 2 bits a la izquierda).
- En el código máquina final, la rotación guardada será 15 (ya que la CPU multiplica la rotación por 2: $15 \times 2 = 30$ bits de rotación derecha) y el valor base será 75.

5. Ejercicios Propuestos

1. Determina qué instrucción en ensamblador se obtiene a partir del código máquina en hexadecimal $0xE04F2003$.
2. Calcula el valor decimal de la constante representada en **Operand2** si los bits de rotación son 0010 (rotación base 2, real $2 \times 2 = 4$) y el valor base es 11110000 (decimal 240).

3. Explica la ventaja estructural de que todas las instrucciones del procesador tengan exactamente el mismo tamaño (32 bits) frente a las instrucciones de tamaño variable en la arquitectura x86.

Tema 4: Modos de Direccionamiento en ARM

Los modos de direccionamiento definen cómo el procesador calcula la ubicación del operando que va a utilizar en una instrucción. ARM destaca por ofrecer una gran flexibilidad en el direccionamiento de datos en registros (gracias a su Barrel Shifter integrado) y en el direccionamiento a memoria RAM (a través de opciones avanzadas de indexación).

1. Direccionamiento a Registros y Escalado (Barrel Shifter)

En las operaciones de procesamiento de datos, el operando secundario (**Operand2**) puede ser un registro modificado de forma transparente por el hardware del **Barrel Shifter** antes de pasar a la ALU, todo en un único ciclo de reloj.

- **LSL (Logical Shift Left)**: Desplazamiento lógico a la izquierda (multiplica por potencias de 2). Rellena con ceros.
- **LSR (Logical Shift Right)**: Desplazamiento lógico a la derecha (divide enteros sin signo entre potencias de 2).
- **ASR (Arithmetic Shift Right)**: Desplazamiento aritmético a la derecha. Preserva el bit de signo (bit 31), ideal para división de enteros con signo.
- **ROR (Rotate Right)**: Rotación de bits a la derecha (los bits que salen por la derecha vuelven a entrar por la izquierda).

Ejemplo de sintaxis:

```
ADD R0, R1, R2, LSL #3    ; R0 = R1 + (R2 * 8)
```

2. Modos de Direccionamiento a Memoria (LDR / STR)

Debido a la arquitectura Load/Store de ARM, el acceso a memoria se restringe a las instrucciones de carga (**LDR**) y almacenamiento (**STR**). Existen tres métodos principales para calcular la dirección de memoria efectiva a partir de un registro base (**Rn**) y un desplazamiento (*offset*):

1. Indexado Directo (Offset Addressing)

Se calcula la dirección sumando o restando el desplazamiento al registro base, pero el registro base **no se modifica** al final de la instrucción.

- *Sintaxis*: **LDR R0, [R1, #4]**
- *Operación*: Carga en **R0** el dato situado en la dirección **R1 + 4**. El registro **R1** permanece inalterado.

2. Pre-indexado (Pre-indexed Addressing)

Se calcula la dirección de acceso sumando el desplazamiento al registro base y, tras acceder a la memoria, el registro base **se actualiza** con la dirección calculada (se escribe el sufijo de *writeback* !).

- *Sintaxis*: **LDR R0, [R1, #4]!**
- *Operación*: Carga en **R0** el dato de la dirección **R1 + 4**, y actualiza **R1 = R1 + 4**.

3. Post-indexado (Post-indexed Addressing)

Se accede a la memoria utilizando **directamente** el valor inicial del registro base. Tras realizar el acceso, el registro base **se actualiza** de forma automática sumándole el desplazamiento.

- *Sintaxis*: **LDR R0, [R1], #4**
- *Operación*: Carga en **R0** el dato situado en la dirección **R1**, y posteriormente actualiza **R1 = R1 + 4**.

3. El Toque Informático

Simulador de Modos de Direccionamiento y Memoria

El siguiente script en Python simula el comportamiento de una memoria RAM virtual y demuestra cómo la CPU procesa los diferentes modos de indexación (directo, pre-indexado con writeback y post-indexado):

```
class MemoriaCPU:
    def __init__(self):
        # Memoria RAM vacía estructurada como diccionario
        self.ram = {0x1000: 99, 0x1004: 150, 0x1008: 200}
        self.r0 = 0
        self.r1 = 0x1000 # Registro base

    def reset_r1(self):
        self.r1 = 0x1000
        self.r0 = 0

    def simular_indexado_directo(self, offset):
        direccion_efectiva = self.r1 + offset
        self.r0 = self.ram.get(direccion_efectiva, 0)
        print("LDR R0, [R1, #offset]")
        print(f" - Dirección de acceso: 0x{direccion_efectiva:04X}")
        print(f" - Valor cargado en R0: {self.r0}")
        print(f" - Estado final R1: 0x{self.r1:04X} (SIN MODIFICAR)")
        print("-" * 55)

    def simular_pre_indexado(self, offset):
        direccion_efectiva = self.r1 + offset
        self.r0 = self.ram.get(direccion_efectiva, 0)
        self.r1 = direccion_efectiva # Writeback activo (!)
        print("LDR R0, [R1, #offset]!")
        print(f" - Dirección de acceso: 0x{direccion_efectiva:04X}")
        print(f" - Valor cargado en R0: {self.r0}")
        print(f" - Estado final R1: 0x{self.r1:04X} (ACTUALIZADO)")
        print("-" * 55)

    def simular_post_indexado(self, offset):
        direccion_efectiva = self.r1
        self.r0 = self.ram.get(direccion_efectiva, 0)
        self.r1 = self.r1 + offset # Actualización posterior
        print("LDR R0, [R1], #offset")
        print(f" - Dirección de acceso: 0x{direccion_efectiva:04X}")
        print(f" - Valor cargado en R0: {self.r0}")
        print(f" - Estado final R1: 0x{self.r1:04X} (ACTUALIZADO POST)")
        print("-" * 55)

# Ejecución de simulación
sim = MemoriaCPU()
sim.simular_indexado_directo(4)

sim.reset_r1()
sim.simular_pre_indexado(4)

sim.reset_r1()
sim.simular_post_indexado(4)
```

4. Ejercicios Resueltos

Ejercicio 1

Dado el estado de registros $R0 = 0x00000000$, $R1 = 0x00002000$ y sabiendo que la posición de memoria $0x00002004$ contiene el valor $0x12345678$, determina el estado final de $R0$ y $R1$ tras ejecutar la instrucción: `LDR R0, [R1, #4]!`

Solución:

1. **Identificar el modo:** Contiene los corchetes que engloban al desplazamiento e incluye el signo de exclamación al final (!), por lo que es un modo **Pre-indexado con actualización (Writeback)**.
2. **Calcular dirección de acceso:**

$$\text{Dirección} = R1 + 4 = 0x2000 + 4 = 0x2004$$

3. **Realizar el acceso y cargar:** Se lee el valor de $0x2004$ ($0x12345678$) y se escribe en el registro destino $R0$.

$$R0 = 0x12345678$$

4. **Actualizar base:** Al ser pre-indexado con !, el registro base $R1$ se actualiza con la dirección calculada:

$$R1 = 0x00002004$$

Resultados finales: $R0 = 0x12345678$, $R1 = 0x00002004$.

Ejercicio 2

Explicar qué utilidad práctica tiene el modo post-indexado (`LDR R0, [R1], #4`) a la hora de procesar elementos consecutivos de un vector.

Solución: En un bucle que procesa un vector de enteros (donde cada entero ocupa 4 bytes), el modo post-indexado realiza dos tareas en una sola instrucción de máquina:

1. Carga el elemento actual apuntado por el puntero $R1$ en $R0$ para procesarlo.
2. Desplaza de forma automática el puntero $R1$ sumándole 4 para que apunte directamente al siguiente entero del vector.

Esto reduce el número de instrucciones del bucle (evitando tener que añadir un `ADD R1, R1, #4` posterior), optimizando la velocidad del bucle y el uso de la memoria de programa.

5. Ejercicios Propuestos

1. Dada la instrucción `STR R4, [R5], #-8`, describe qué operación se realiza y cómo queda el valor de `R5` tras el almacenamiento.
2. Escribe la instrucción equivalente para calcular: $R0 = R1 - (R2 \cdot 16)$ utilizando el Barrel Shifter en una única instrucción.
3. Si ejecutas `LDR R0, [R1, R2, LSL #2]`, explica paso a paso cómo calcula el procesador la dirección de acceso a memoria. ¿Cómo se denomina este modo de direccionamiento?

Tema 5: Procesamiento de Datos y Control de Secuenciamiento en ARM

El núcleo funcional de cualquier CPU es la capacidad de procesar datos (operaciones aritméticas y lógicas) y alterar el flujo de ejecución del programa en función de las condiciones resultantes. La arquitectura ARM integra un sistema de ejecución condicional de instrucciones único en la industria.

1. Instrucciones de Procesamiento de Datos

Las instrucciones de procesamiento de datos operan exclusivamente en registros internos. Se pueden clasificar en:

Operaciones Aritméticas

- **ADD (Addition)**: Suma básica de dos operandos: `ADD R0, R1, R2` ($R0 = R1 + R2$).
- **SUB (Subtraction)**: Resta básica: `SUB R0, R1, R2` ($R0 = R1 - R2$).
- **RSB (Reverse Subtraction)**: Resta inversa: `RSB R0, R1, R2` ($R0 = R2 - R1$). Útil para restar registros de valores inmediatos.
- **ADC / SBC (Addition/Subtraction with Carry)**: Suma o resta incorporando el bit de acarreo (C) del registro de estado CPSR. Permite realizar operaciones de precisión múltiple (ej. operar números de 64 bits).

Operaciones Lógicas y de Manipulación de Bits

- **AND**: Operación Y lógica bit a bit: `AND R0, R1, R2` ($R0 = R1 \wedge R2$).
- **ORR**: Operación O lógica inclusiva: `ORR R0, R1, R2` ($R0 = R1 \vee R2$).
- **EOR**: Operación O lógica exclusiva (XOR): `EOR R0, R1, R2` ($R0 = R1 \oplus R2$).
- **BIC (Bit Clear)**: Limpia los bits del primer operando que están a 1 en el segundo operando: `BIC R0, R1, R2` ($R0 = R1 \wedge \neg R2$).

Operaciones de Comparación (no guardan resultado en Rd, solo actualizan flags)

- **CMP (Compare):** Resta implícita ($Rn - \text{Operand2}$) que actualiza las banderas del CPSR.
- **TST (Test):** Y lógica implícita ($Rn \wedge \text{Operand2}$) que actualiza las banderas del CPSR.

2. Ejecución Condicional en ARM

En la mayoría de las arquitecturas clásicas, la ejecución condicional se limita únicamente a las instrucciones de salto. En ARM, **casi todas las instrucciones de procesamiento de datos y transferencia pueden ser condicionales** mediante la adición de un sufijo de dos letras que se corresponde con el estado de los flags del CPSR.

Sufijo	Nombre	Flags Evaluados	Significado Aritmético
EQ	Equal	$Z = 1$	Igualdad / Resultado cero
NE	Not Equal	$Z = 0$	Desigualdad / Resultado no cero
CS / HS	Carry Set / Higher or Same	$C = 1$	Mayor o igual (sin signo)
CC / LO	Carry Clear / Lower	$C = 0$	Menor (sin signo)
MI	Minus	$N = 1$	Negativo
PL	Plus	$N = 0$	Positivo o cero
GT	Greater Than	$Z = 0$ y $N = V$	Mayor que (con signo)
LT	Less Than	$N \neq V$	Menor que (con signo)
GE	Greater or Equal	$N = V$	Mayor o igual que (con signo)
LE	Less or Equal	$Z = 1$ o $N \neq V$	Menor o igual que (con signo)

Ejemplo práctico (evitando saltos):

- En C:

```
if (R1 == R2) R0 = R0 + 1;
else R0 = R0 - 1;
```

- En Ensamblador ARM sin ejecución condicional en datos (usando saltos):

```
CMP R1, R2
BNE sino
ADD R0, R0, #1
B fin
sino:
    SUB R0, R0, #1
fin:
```

- En Ensamblador ARM optimizado (aprovechando la ejecución condicional):

```
CMP R1, R2
ADDEQ R0, R0, #1
SUBNE R0, R0, #1
```

Esta optimización elimina las penalizaciones por desalineación de cauce (*pipeline stalls*) provocadas por saltos condicionales en el procesador.

3. Instrucciones de Salto (Control de Secuenciamiento)

- **B (Branch)**: Salto condicional o incondicional a una etiqueta relativa: **B** *etiqueta* o **BNE** *bucle*.
- **BX (Branch and Exchange)**: Salto a una dirección contenida en un registro, permitiendo opcionalmente cambiar de estado de ejecución (ARM/Thumb): **BX** *LR*.

4. El Toque Informático

Simulador de Ejecución Condicional de la ALU

El siguiente script en Python simula cómo la Unidad de Control de la CPU evalúa los sufijos condicionales leyendo el CPSR antes de permitir que la ALU ejecute una instrucción:

```
class ALUConCondicionales:
    def __init__(self):
        # Flags CPSR simulados
        self.flags = {"N": 0, "Z": 0, "C": 0, "V": 0}
        self.r0 = 100

    def evaluar_condicion(self, sufijo):
        z, n, c, v = self.flags["Z"], self.flags["N"], self.flags["C"], self.flags["V"]
        if sufijo == "AL": # Always
            return True
        elif sufijo == "EQ": # Equal
            return z == 1
        elif sufijo == "NE": # Not Equal
            return z == 0
        elif sufijo == "GT": # Greater Than (signed)
            return z == 0 and n == v
        elif sufijo == "LT": # Less Than (signed)
            return n != v
        return False

    def ejecutar_instruccion(self, cond_sufijo, op_name, valor):
        # Primero evaluamos si se cumple la condición
        debe_ejecutar = self.evaluar_condicion(cond_sufijo)

        if debe_ejecutar:
            if op_name == "ADD":
                self.r0 += valor
            elif op_name == "SUB":
                self.r0 -= valor
            print(f"Ejecutando: {op_name}{cond_sufijo} #{valor} -> R0 = {self.r0}")
        else:
            print(f"Omitiendo: {op_name}{cond_sufijo} #{valor} (Condición no cumplida)")

# Simulación
alu = ALUConCondicionales()
# Simulamos un resultado previo donde R1 == R2 (Z = 1)
alu.flags["Z"] = 1

# Intentamos ejecutar dos instrucciones consecutivas
alu.ejecutar_instruccion("EQ", "ADD", 10) # Debería ejecutarse
alu.ejecutar_instruccion("NE", "SUB", 5) # Debería omitirse
```

5. Ejercicios Resueltos

Ejercicio 1

Dadas las instrucciones:

```
CMP R0, R1
SUBGT R2, R2, #10
```

Si $R0 = 5$ y $R1 = 8$, explica paso a paso qué ocurre durante la ejecución de la instrucción **SUBGT**.

Solución:

1. **CMP R0, R1**: Realiza la operación $R0 - R1 \Rightarrow 5 - 8 = -3$.
 - Como el resultado es diferente de cero, $Z = 0$.
 - Como el resultado es negativo y no hay desbordamiento de signo, $N = 1$ y $V = 0$.
2. **SUBGT R2, R2, #10**: La instrucción tiene el sufijo **GT** (Greater Than - Mayor que con signo).
 - La CPU evalúa la condición **GT**, que requiere que: $Z = 0$ y $N = V$.
 - Comprobamos los flags: $Z = 0$ (se cumple) pero $N = 1$ y $V = 0 \Rightarrow N \neq V$ (no se cumple).
 - Como la condición no es verdadera, la CPU **omite** la resta y actúa como un **NOP** (No Operación). El registro **R2** mantiene su valor original.

Ejercicio 2

Explicar el funcionamiento de la instrucción **BIC R0, R1, #0xFF** e indicar cómo altera los bits de **R1**.

Solución: La instrucción **BIC** (*Bit Clear*) borra los bits del primer registro fuente (**R1**) correspondientes a las posiciones donde el segundo operando (máscara) tiene un valor de **1**.

- El operando **#0xFF** en binario de 32 bits es: **0x000000FF** (los 8 bits menos significativos están a 1, el resto a 0).
- La operación lógica es: $R0 = R1 \wedge \neg(0x000000FF)$.
- Por lo tanto, **BIC** pone a cero los 8 bits menos significativos de **R1** y mantiene intactos los 24 bits más significativos. El resultado final se guarda en **R0**.

6. Ejercicios Propuestos

1. Dada la siguiente sección de código en C:

```
if (a < b) {  
    c = c + 5;  
} else {  
    c = c - 5;  
}
```

Escribe su traducción equivalente en ensamblador ARM optimizando el código para evitar instrucciones de salto, asumiendo que **a** está en **R0**, **b** en **R1** y **c** en **R2**.

2. Investiga el comportamiento de la instrucción **RSBS R0, R1, #0**. ¿Qué operación aritmética está realizando sobre **R1** y cuál es su efecto sobre las banderas del CPSR?
3. Explica la diferencia entre las condiciones de comparación **sin signo** (ej: **HI**, **LO**) y **con signo** (ej: **GT**, **LT**). ¿Por qué la condición **GE** requiere evaluar el flag **V** (Overflow) además del flag **N**?

Tema 6: Traducción de Estructuras de Control a Ensamblador ARM

En la programación ordinaria en lenguajes de alto nivel, el programador emplea condicionales (**if-else**) y bucles (**for**, **while**) para guiar la ejecución. Sin embargo, a nivel de máquina no existen tales construcciones complejas. La CPU ejecuta instrucciones de manera secuencial a menos que se fuerce una alteración explícita de la secuencia mediante saltos y comparaciones.

1. Traducción de Condicionales (if-else)

La estructura condicional **if-else** se traduce utilizando una combinación de comparación (**CMP**), saltos condicionales y etiquetas de salto.

Estructura General

- En C:

```
if (condicion) {  
    // Bloque IF  
} else {  
    // Bloque ELSE  
}
```

- En Ensamblador ARM (Estructura clásica):

```
CMP R0, R1          ; Comparación de variables  
B_condicion_falsa else_label  
; --- Bloque IF ---  
; código si es verdadero  
B fin_if  
else_label:  
; --- Bloque ELSE ---  
; código si es falso  
fin_if:
```

2. Traducción de Bucles (while y for)

Los bucles se implementan ejecutando repetidamente un bloque de código y realizando una comparación condicional al final o al inicio de cada iteración.

El Bucle **while** (Pre-test)

- En C:

```
while (R0 < 10) {  
    R0++;  
}
```

- En Ensamblador ARM:

```
bucle_while:  
    CMP R0, #10          ; Evaluar condición  
    BGE fin_while        ; Si R0 >= 10, salimos del bucle  
    ADD R0, R0, #1       ; Cuerpo del bucle: R0 = R0 + 1  
    B bucle_while        ; Volver a evaluar  
fin_while:
```

El Bucle **for** (Contador)

- En C:

```
for (int i = 0; i < 100; i++) {  
    suma += i;  
}
```

- En Ensamblador ARM (Asumiendo i en R0, suma en R1): **MOV R0, #0**
(Inicialización de **i = 0**).

```
bucle_for:  
    CMP R0, #100         ; Comparar i con 100  
    BGE fin_for          ; Si i >= 100, salir  
    ADD R1, R1, R0       ; suma = suma + i  
    ADD R0, R0, #1       ; i++
```

```
B bucle_for          ; Siguiente iteración
fin_for:
```

3. Vectores, Tablas y Direcccionamiento de Arrays

Para manipular un array en ensamblador, debemos calcular la dirección de memoria exacta de cada elemento en función de su índice y del tamaño físico del tipo de datos (en bytes):

Dirección del elemento i = Dirección Base del Array + ($i \cdot$ Tamaño en bytes del elemento)

- **Tipos de datos de tamaño fijo:**
 - **char** (1 byte): Desplazamiento por índice directo.
 - **short** (2 bytes): Desplazamiento por **índice * 2**.
 - **int** o **float** (4 bytes): Desplazamiento por **índice * 4**.

Bucle de recorrido de un vector en ARM (con post-indexado)

- En C:

```
int vector[5] = {10, 20, 30, 40, 50};
int suma = 0;
for (int i = 0; i < 5; i++) {
    suma += vector[i];
}
```

- En Ensamblador ARM (Asumiendo R1 = Puntero a vector, R2 = suma, R3 = contador i): **MOV R2, #0** (suma = 0), **MOV R3, #5** (contador de iteraciones = 5).

```
bucle_vector:
    SUBS R3, R3, #1      ; Decrementar contador y actualizar flags
    BMI fin_vector      ; Si R3 < 0, salir del bucle
    LDR R0, [R1], #4     ; Cargar elemento actual y avanzar puntero (Post-indexado)
    ADD R2, R2, R0       ; suma += elemento
    B bucle_vector
fin_vector:
```

4. El Toque Informático

Traductor de Bucles de Alto Nivel a Ensamblador ARM

El siguiente script en Python simula un mini-compilador que recibe las propiedades de un bucle `for` y genera de forma automática el bloque correspondiente en código ensamblador ARMv7:

```
def generar_codigo_bucle_arm(var_ind, limite, paso, registro_indice, registro_suma):
    # Genera la estructura de código ensamblador para un bucle for clásico
    print(f"; --- Bucle generado automáticamente para variable '{var_ind}' ---")
    print(f"      MOV {registro_indice}, #0          ; Inicializar {var_ind} = 0")
    print("bucle_for:")
    print(f"      CMP {registro_indice}, #{limite}      ; Comparar con límite")
    print(f"      BGE fin_for                          ; Si es mayor o igual, salir")
    print(f"      ADD {registro_suma}, {registro_suma}, {registro_indice} ; Sumar valor")
    print(f"      ADD {registro_indice}, {registro_indice}, #{paso}      ; Incremento")
    print(f"      B bucle_for                          ; Siguiendo ciclo")
    print("fin_for:")
    print("; -----")

# Generar bucle: for (int i = 0; i < 10; i++)
generar_codigo_bucle_arm("i", 10, 1, "R0", "R1")
```

5. Ejercicios Resueltos

Ejercicio 1

Traducir a ensamblador ARM la siguiente estructura condicional en alto nivel:

```
if (R0 > 25) {
    R1 = R1 + 10;
} else {
    R1 = R1 - 10;
}
```

Solución: Podemos hacerlo mediante dos enfoques:

1. Enfoque de Salto Clásico (Ideal para bloques de código largos):

```
CMP R0, #25      ; Comparar R0 con 25
BLE else_block   ; Si R0 <= 25, saltar al bloque else
ADD R1, R1, #10   ; Bloque if: R1 = R1 + 10
B end_if
else_block:
SUB R1, R1, #10   ; Bloque else: R1 = R1 - 10
end_if:
```

2. Enfoque de Ejecución Condicional (Ideal para optimización de pocas líneas):

```
CMP R0, #25      ; Comparar R0 con 25
ADDGT R1, R1, #10 ; R1 = R1 + 10 si R0 > 25
SUBLE R1, R1, #10 ; R1 = R1 - 10 si R0 <= 25
```

Ejercicio 2

Un programador tiene la base de un vector de enteros en **R0** y quiere acceder al elemento **vector[3]**. Describe la instrucción ensamblador optimizada para cargar este elemento en **R1** en una única instrucción aprovechando los modos de direccionamiento.

Solución:

- Cada elemento **int** del vector ocupa 4 bytes en memoria.
- El elemento **vector[3]** se sitúa en un desplazamiento de $3 \times 4 = 12$ bytes desde la dirección base en **R0**.
- La instrucción óptima es: **LDR R1, [R0, #12]** Esta instrucción suma 12 a **R0**, lee la posición resultante en la memoria RAM y almacena el entero de 32 bits cargado en **R1**. El puntero base **R0** no se modifica.

6. Ejercicios Propuestos

1. Escribe el código ensamblador ARM equivalente al siguiente bucle **while**:

```
while (R1 != 0) {
    R2 = R2 + 2;
    R1 = R1 - 1;
}
```

2. Dado un vector de bytes (**char**) cuya dirección base está en **R1**, escribe las instrucciones necesarias para recorrer e inicializar los primeros 10 elementos a cero.
3. Explica qué problema de rendimiento ocurre en los microprocesadores modernos con cauce segmentado (pipelined CPUs) al procesar instrucciones de salto (**B**) y por qué la ejecución condicional de datos ayuda a solventarlo.

Tema 7: El Mecanismo de Llamada a Subrutinas y el Link Register (LR)

La modularidad es un principio básico del desarrollo de software que consiste en dividir un programa complejo en subprogramas reutilizables denominados funciones, procedimientos o subrutinas. A nivel de hardware, esto requiere de un mecanismo de salto bidireccional: la CPU debe saltar al inicio de la subrutina y, al finalizar su ejecución, debe retornar exactamente a la instrucción posterior a la llamada.

1. El Protocolo de Llamada y Retorno: La Instrucción BL y el Registro LR

Para realizar la llamada a una subrutina, ARM dispone de la instrucción dedicada **BL** (Branch with Link).

- **Funcionamiento de BL subrutina:**
 1. El procesador guarda de forma automática en el **Link Register (LR / R14)** la dirección de retorno (la dirección de la instrucción inmediatamente posterior a la llamada **BL**).
 2. Actualiza el **Program Counter (PC / R15)** con la dirección de la subrutina para transferir el control de ejecución.
 - **Retorno de la subrutina:** Para regresar, la subrutina debe transferir el valor almacenado en **LR** de vuelta al contador de programa **PC**. Se puede hacer mediante dos instrucciones equivalentes:
 - **MOV PC, LR**
 - **BX LR** (Branch and Exchange, preferible porque permite interconexión Thumb).
-

2. El Problema de las Subrutinas Anidadas

Una subrutina se denomina "**Hoja**" (**Leaf Subroutine**) si no realiza llamadas a ninguna otra subrutina. En este caso, puede mantener el registro **LR** intacto durante toda su ejecución y retornar directamente usándolo.

Sin embargo, cuando una subrutina llama a otra (subrutina **"No Hoja"** o llamadas anidadas), ocurre una sobreescritura de **LR**:

```
Principal:
  BL FuncionA  --> [LR = Dirección Retorno a Principal]
                Jumps to FuncionA...

FuncionA:
  ...
  BL FuncionB  --> [LR = Dirección Retorno a FuncionA (¡Sobreescrito!)]
                Jumps to FuncionB...

FuncionB:
  ...
  BX LR        --> Retorna a FuncionA (Carga LR en PC)

FuncionA:
  ...
  BX LR        --> ¡Bucle infinito! (Intenta retornar a FuncionA porque LR fue
sobreescribio)
```

La Solución: Preservación de LR en el Stack (Pila)

Para evitar la pérdida de la dirección de retorno a la función superior, toda subrutina no hoja debe salvar **LR** al entrar en la pila y recuperarlo al salir:

```
FuncionA:
  PUSH {LR}      ; Guardar LR en la pila al entrar (Inicio de la función)
  ...
  BL FuncionB    ; LR se sobrescribe con el retorno a FuncionA
  ...
  POP {PC}       ; Extrae el valor guardado de la pila directamente a PC
                  ; (Equivale a un POP de LR y luego MOV PC, LR, ahorrando un ciclo)
```

3. El Toque Informático

Simulador del Flujo de Ejecución y Link Register

El siguiente script en Python simula el contador de programa **PC** y el registro **LR** de una CPU durante una secuencia de llamadas anidadas, ilustrando la corrupción del retorno si no se aplica la preservación en el stack:

```

class CallStackSimulator:
    def __init__(self):
        self.pc = 0x0100
        self.lr = 0x0000
        self.stack = []

    def llamar_subrutina(self, destino_addr, nombre, preservar=False):
        retorno_addr = self.pc + 4
        self.lr = retorno_addr
        if preservar:
            self.stack.append(retorno_addr)
            print(f"Llamando a {nombre}: Guardado retorno 0x{retorno_addr:04X} en la
Pila.")
        else:
            print(f"Llamando a {nombre}: LR actualizado con retorno 0x{retorno_addr:04X}
(SIN PILA).")
            self.pc = destino_addr

    def retornar_subrutina(self, usar_pila=False):
        if usar_pila:
            if len(self.stack) > 0:
                retorno = self.stack.pop()
                self.pc = retorno
                print(f"Retornando desde Pila: PC actualizado a 0x{self.pc:04X}")
            else:
                print("[ERROR] Pila vacía. Crash del sistema.")
        else:
            self.pc = self.lr
            print(f"Retornando desde LR: PC actualizado a 0x{self.pc:04X}")

# Simulación de llamadas anidadas correctas (usando la Pila)
sim = CallStackSimulator()
print("--- SIMULACIÓN DE LLAMADAS CORRECTAS ---")
sim.llamar_subrutina(0x2000, "FuncionA", preservar=True)
sim.pc = 0x2010 # Avance de programa en FuncionA
sim.llamar_subrutina(0x3000, "FuncionB", preservar=True)
sim.pc = 0x3008 # Fin de FuncionB
sim.retornar_subrutina(usar_pila=True) # Retorna a FuncionA
sim.pc = 0x2018 # Fin de FuncionA
sim.retornar_subrutina(usar_pila=True) # Retorna a Principal
print("-" * 55)

```

4. Ejercicios Resueltos

Ejercicio 1

Escribir el esqueleto básico en ensamblador ARM para una subrutina llamada **CalcularFactorial** que a su vez llama internamente a una subrutina multiplicadora **MultiplicarEnteros**, aplicando el protocolo de llamada y retorno seguro.

Solución:

```
CalcularFactorial:
    PUSH {R4, LR}          ; Preservamos registros de trabajo y el registro LR
    ; ... cuerpo de la función ...
    ; aquí preparamos argumentos en R0 y R1
    BL MultiplicarEnteros ; Llamada interna (sobreescribe LR)
    ; R0 contiene el resultado del factorial temporal
    ; ... más procesamiento ...
    POP {R4, PC}           ; Restauramos R4 y cargamos la dirección de retorno guardada en
PC
```

Ejercicio 2

Explicar por qué es posible retornar de una subrutina hoja utilizando **BX LR** sin necesidad de realizar accesos a la pila de memoria RAM.

Solución: Dado que la subrutina hoja no realiza llamadas internas a otras funciones, el hardware de la CPU no modifica el valor inicial de **LR** durante toda la ejecución del subprograma. Por lo tanto, el valor correcto de la dirección de retorno a la función llamadora permanece intacto en el registro **LR**. Al ser **LR** un registro interno del procesador, retornar mediante **BX LR** evita accesos costosos a la memoria RAM (lecturas del stack), ejecutando la vuelta de forma instantánea.

5. Ejercicios Propuestos

1. Dada la instrucción **BL etiqueta**, indica en qué dirección de memoria exacta se calcula el retorno si la instrucción **BL** se sitúa en la posición **0x00008000**.
2. Un programador escribe una subrutina recursiva. Describe detalladamente qué ocurrirá con la pila de memoria (Stack) y con el registro **LR** si se olvida de implementar la salvaguarda de **LR** en el prólogo de la función.
3. Investiga el estándar EABI/AAPCS de ARM. ¿Qué registros se consideran volátiles (caller-saved) y cuáles deben ser obligatoriamente restaurados por la subrutina al finalizar (callee-saved)?

Tema 8: La Pila (Stack) y la Preservación del Contexto en ARM

La pila (*Stack*) es una estructura de datos de tipo LIFO (Last-In, First-Out: el último elemento en entrar es el primero en salir) gestionada directamente por el hardware del procesador mediante el registro **Stack Pointer (SP / R13)**. La pila es vital para implementar subrutinas, almacenar variables locales y preservar el estado interno de los registros de la CPU.

1. Clasificación de Pilas y el Estándar de ARM

Una pila en memoria se puede clasificar según dos criterios independientes:

Según la dirección de crecimiento de la memoria:

- **Ascendente (Ascending)**: La pila crece hacia direcciones de memoria más altas (se suma al SP al apilar).
- **Descendente (Descending)**: La pila crece hacia direcciones de memoria más bajas (se resta al SP al apilar).

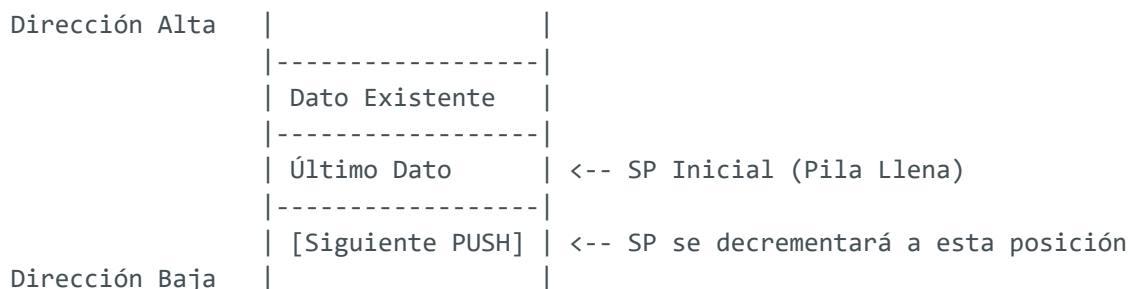
Según a dónde apunta el puntero SP:

- **Vacía (Empty)**: **SP** apunta a la primera posición vacía (libre) de la pila.
- **Llena (Full)**: **SP** apunta al último elemento realmente almacenado en la pila.

El Estándar ARM: Pila Llena Descendente (Full Descending - FD)

Por convención, el estándar de llamadas de la arquitectura ARM (AAPCS) establece que la pila es **Full Descending (FD)**:

- Al realizar un **PUSH (apilar)**: El puntero **SP** se decrementa primero en 4 bytes y luego se escribe el dato en la dirección resultante.
- Al realizar un **POP (desapilar)**: Se lee el dato de la dirección actual apuntada por **SP** y posteriormente el puntero **SP** se incrementa en 4 bytes.



2. Instrucciones PUSH y POP en ARM

ARM no dispone de instrucciones de apilado unitario elementales. En su lugar, utiliza instrucciones de transferencia múltiple extremadamente potentes, que admiten alias nemotécnicos:

- **PUSH {R4, R5, LR}**: Apila el conjunto de registros especificado. Su instrucción máquina equivalente en pila FD es: **STMDB SP!, {R4, R5, LR}** (*Store Multiple Decrement Before* - Decrementa el puntero antes de almacenar).
- **POP {R4, R5, PC}**: Desapila los registros y transfiere el retorno directamente al contador de programa. Su equivalente físico es: **LDMIA SP!, {R4, R5, PC}** (*Load Multiple Increment After* - Lee la memoria e incrementa el puntero después).

Nota: El signo de exclamación ! indica actualización obligatoria (writeback) del registro base SP tras la operación.

3. Preservación del Contexto

Para que las subrutinas no interfieran entre sí, se define un acuerdo de preservación de registros (contexto del procesador):

- **Registros no volátiles (callee-saved)**: Los registros **R4-R11** y **SP** deben conservar su valor al retornar de la función. Si la subrutina necesita utilizarlos, debe apilarlos en el prólogo y restaurarlos en el epílogo.

- **Registros volátiles (caller-saved):** **R0-R3**, **R12** y **LR** pueden ser destruidos libremente por la subrutina.

MiSubrutina:

```
PUSH {R4, R5, R11, LR} ; Prólogo: Guardar contexto y dirección de retorno  
; ... cuerpo de la subrutina utilizando R4 y R5 ...  
POP {R4, R5, R11, PC} ; Epílogo: Restaurar contexto y retornar
```

4. El Toque Informático

Simulador de Pila Llena Descendente (Full Descending Stack)

El siguiente script en Python simula el comportamiento detallado del mapa de memoria RAM de la pila y del puntero **SP** durante operaciones consecutivas de PUSH y POP:

```
class StackSimulator:  
    def __init__(self):  
        self.ram = {}  
        self.sp = 0x8000 # Dirección de inicio alta de la pila  
  
    def push(self, valor):  
        # En pila FD: Primero decrementar SP, luego guardar  
        self.sp -= 4  
        self.ram[self.sp] = valor  
        print(f"PUSH {valor} -> SP se decrementa a 0x{self.sp:04X} | Guardado  
[{valor}]]")  
        self.print_stack()  
  
    def pop(self):  
        # En pila FD: Primero leer de SP, luego incrementar SP  
        if self.sp >= 0x8000:  
            print("[ERROR] Stack Underflow: La pila está vacía.")  
            return None  
        valor = self.ram[self.sp]  
        del self.ram[self.sp] # Limpiar simulación  
        self.sp += 0x0004  
        print(f"POP -> Leído [{valor}] desde SP | SP se incrementa a 0x{self.sp:04X}")  
        self.print_stack()  
        return valor  
  
    def print_stack(self):  
        print("ESTADO ACTUAL DE LA PILA:")  
        for addr in sorted(self.ram.keys(), reverse=True):  
            marker = " <-- SP" if addr == self.sp else ""  
            print(f"  [0x{addr:04X}]: {self.ram[addr]}{marker}")  
        if self.sp == 0x8000:
```

```
print(" [0x8000]: (CIMA DE PILA VACÍA) <-- SP")
print("-" * 55)
```

```
# Ejecución
pila = StackSimulator()
pila.push(42)
pila.push(99)
pila.pop()
pila.pop()
```

5. Ejercicios Resueltos

Ejercicio 1

Dado un estado inicial de la pila con $SP = 0x00007FC0$. Si ejecutamos la instrucción **PUSH** $\{R0, R1, R2\}$, determina:

1. El valor final del puntero de pila **SP**.
2. Las direcciones de memoria exactas ocupadas por cada uno de los registros en una pila Full Descending.

Solución:

1. Cálculo del SP final:

- La instrucción apila 3 registros (**R0**, **R1** y **R2**). Cada registro ocupa 4 bytes.
- El tamaño total es $3 \times 4 = 12$ bytes.
- Al ser descendente:

$$SP \text{ final} = 0x7FC0 - 12 = 0x7FC0 - 0x000C = 0x00007FB4$$

- ##### 2. Distribución de registros en memoria:
- En ARM, las instrucciones de almacenamiento múltiple (**STM**) guardan los registros en memoria de forma que **el registro con menor índice se almacena en la dirección más baja** y el de mayor índice en la más alta:

- Dirección **0x00007FB4**: Guarda **R0** (Dirección final de **SP**).
- Dirección **0x00007FB8**: Guarda **R1**.
- Dirección **0x00007FBC**: Guarda **R2**.

Ejercicio 2

Explicar qué ocurre si dentro de una subrutina realizamos un **PUSH {R4, LR}** pero al finalizar ejecutamos un **POP {R4, LR}** seguido de **BX LR**. ¿Es correcto? ¿Existe alguna alternativa más eficiente?

Solución: La secuencia es funcionalmente correcta: se guarda el contexto y retorno, se restaura y se salta a **LR**. Sin embargo, es **ineficiente** porque requiere dos instrucciones de control de flujo al final. La alternativa estándar de la arquitectura ARM consiste en hacer: **POP {R4, PC}** Esta instrucción desapila el contenido original de **R4** de vuelta a **R4** y, al mismo tiempo, extrae la dirección de retorno que estaba en la pila y la carga directamente en el Program Counter (**PC**). Esto realiza la restauración y el salto de retorno en un único ciclo de instrucción máquina.

6. Ejercicios Propuestos

1. Diferencia razonadamente el comportamiento de una pila **Full Descending (FD)** frente a una pila **Empty Descending (ED)** al realizar una operación de apilado.
2. Dado un programa que realiza un bucle infinito de **PUSH** sin realizar nunca un **POP**, describe qué tipo de fallo de memoria ocurrirá y por qué los sistemas operativos limitan el tamaño de la pila de los programas.
3. Escribe el prólogo y epílogo en ensamblador ARM para una subrutina que utiliza los registros de trabajo **R4**, **R5** y **R6**, asegurando la correcta preservación del contexto y retorno seguro.

Tema 9: Bloques de Activación (Stack Frames) y Gestión de Variables Locales

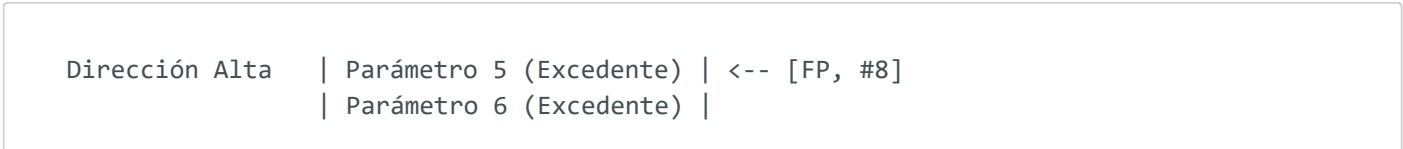
Cuando una subrutina está en ejecución, requiere espacio en memoria para almacenar variables de ámbito local, argumentos de llamada que exceden la capacidad de los registros físicos y el contexto de retorno. El **Bloque de Activación** (o *Stack Frame*) es el registro dinámico de memoria reservado en la pila para dar soporte a una invocación particular de una subrutina.

1. El Rol del Frame Pointer (FP) frente al Stack Pointer (SP)

- **Stack Pointer (SP / R13)**: Puntero de cima de pila. Se modifica continuamente durante la ejecución de la subrutina debido a operaciones de apilado temporal (**PUSH** y **POP**). Esto hace que referenciar una variable local utilizando un desplazamiento fijo respecto a **SP** (ej: `[SP, #8]`) sea inestable y propenso a errores del compilador.
- **Frame Pointer (FP / R11)**: Puntero de marco de pila. Permanece **estático e invariable** a lo largo de toda la vida de la subrutina. Proporciona una base estable para direccionar datos:
 - **Variables locales**: Direccionadas en desplazamientos **negativos** respecto al **FP** (ej: `[FP, #-16]`).
 - **Parámetros excedentes**: Parámetros que no caben en los registros R0-R3 y se pasan en la pila, direccionados en desplazamientos **positivos** respecto al **FP** (ej: `[FP, #8]`).

2. Anatomía de un Bloque de Activación (Pila FD)

La estructura típica de un bloque de activación en ARM tras ejecutar el prólogo de la función sigue este diseño espacial en la pila:



	LR (Retorno original)	<-- [FP, #4]
	FP (FP anterior)	<-- FP Actual (R11) apunta aquí
	R4 (Contexto salvado)	<-- [FP, #-4]

	Variable Local 1	<-- [FP, #-8]
	Variable Local 2	<-- [FP, #-12] (Alineada)

Dirección Baja	Cima de Pila Actual	<-- SP (Modificable)

3. Prólogo y Epílogo de una Función con Stack Frame

Para inicializar y destruir correctamente este bloque, la subrutina ejecuta rutinas fijas de entrada y salida:

El Prólogo (Creación del Bloque)

```
MiFuncion:
    PUSH {R4, FP, LR}      ; 1. Salvar registros y el FP del llamador
    ADD R11, SP, #4        ; 2. Establecer el nuevo FP (apunta a la posición de FP
salvado)
    SUB SP, SP, #8         ; 3. Reservar 8 bytes para variables locales (2 enteros)
```

El Epílogo (Destrucción del Bloque y Retorno)

```
SUB SP, R11, #4           ; 1. Desasignar variables locales (restaurar SP desde FP)
POP {R4, FP, PC}          ; 2. Restaurar registros del llamador, su FP y retornar
```

4. El Toque Informático

Visualizador del Layout de Memoria de un Stack Frame

El siguiente script en Python genera y visualiza en la consola el mapa de direcciones físicas de un bloque de activación dinámico a partir de la declaración de variables locales y parámetros del programador:

```
def mapear_stack_frame(num_variables_locales, num_parametros):
    fp = 0x7FC0
    print("=== MAPA ESPACIAL DEL STACK FRAME ===")
    print(f"Dirección | {'Contenido':25s} | Desplazamiento FP")
    print("-" * 50)

    # 1. Parámetros excedentes (pasados por el llamador)
    for p in range(num_parametros, 4, -1):
        offset = (p - 5) * 4 + 8
        print(f"0x{fp + offset:04X} | Parámetro Excedente {p:02d} | [FP, #{offset}]")

    print(f"0x{fp + 4:04X} | Link Register (LR) | [FP, #4]")
    print(f"0x{fp:04X} | Frame Pointer (FP) anterior | [FP, #0] <-- FP Actual")

    # 2. Contexto y variables locales (direcciones inferiores)
    for v in range(1, num_variables_locales + 1):
        offset = -v * 4
        print(f"0x{fp + offset:04X} | Variable Local {v:02d} | [FP, #{offset}]")

    sp_final = fp - (num_variables_locales * 4)
    print(f"0x{sp_final:04X} | Cima de Pila Actual | <-- SP")
    print("=" * 50)

    # Ejemplo: Función con 3 variables locales de 32 bits y 6 parámetros (2 excedentes)
    mapear_stack_frame(num_variables_locales=3, num_parametros=6)
```

5. Ejercicios Resueltos

Ejercicio 1

Dada una función en C:

```
void MiFuncion(int a, int b, int c, int d, int e, int f) {
    int local1;
    local1 = e + f;
}
```

Escribir el código en ensamblador ARM para acceder a las variables y calcular la suma en `local1`, asumiendo que ya se ha creado el Bloque de Activación.

Solución:

1. Identificar la ubicación de los argumentos:

- Los parámetros **a**, **b**, **c** y **d** se pasan en los registros **R0**, **R1**, **R2** y **R3** respectivamente.
- Los parámetros **e** y **f** son excedentes (parámetros 5 y 6), por lo que se pasan en la pila. Se encuentran en **[FP, #8]** (**e**) y **[FP, #12]** (**f**).

2. Identificar la ubicación de la variable local:

- La variable **local1** se almacena en el primer espacio de variables locales en la pila, direccionado en **[FP, #-8]**.

3. Escribir las instrucciones:

```
LDR R0, [R11, #8]    ; Cargar parámetro 'e' en R0
LDR R1, [R11, #12]   ; Cargar parámetro 'f' en R1
ADD R2, R0, R1       ; R2 = e + f
STR R2, [R11, #-8]   ; Guardar el resultado en 'local1'
```

6. Ejercicios Propuestos

1. Explica razonadamente por qué el estándar de llamadas AAPCS obliga a que la pila del procesador ARM mantenga una **alineación de 8 bytes** de doble palabra al realizar llamadas a subrutinas externas.
2. Un programador decide no usar el registro **FP** (R11) y direccionar todas las variables locales mediante desplazamientos relativos al puntero **SP**. Discute las ventajas e inconvenientes de esta práctica (conocida como *omitir el puntero de marco* o *frame pointer omission*).
3. Dibuena un diagrama en papel de la disposición de memoria de un Bloque de Activación para una función que recibe 5 parámetros y declara 4 variables locales tipo entero de 32 bits.

Tema 10: Interoperabilidad de Lenguajes: Código C y Ensamblador ARM

En el desarrollo de software profesional de sistemas, sistemas empotrados y motores de alto rendimiento, es habitual combinar código escrito en C/C++ con rutinas optimizadas en ensamblador. Para que esta comunicación bidireccional funcione sin corromper el sistema, el procesador sigue una norma estricta: el **Estándar de Llamadas para la Arquitectura ARM (AAPCS / EABI)**.

1. El Estándar de Llamadas de ARM (AAPCS)

El estándar **AAPCS (Procedure Call Standard for the ARM Architecture)** define los convenios que regulan el uso de los registros y de la pila para llamadas a funciones:

Paso de Parámetros y Retorno de Valores

- **Primeros 4 parámetros:** Se colocan obligatoriamente de izquierda a derecha en los registros **R0**, **R1**, **R2** y **R3**.
- **Parámetros excedentes (a partir del 5º):** Se apilan en el Stack (**R13**), en orden inverso (de derecha a izquierda, de modo que el 5º parámetro quede en la cima de la pila).
- **Retorno de valores:** El resultado devuelto por la función se deposita en el registro **R0** (si el dato es de 32 bits o menor) o en la pareja **R0-R1** (si es un entero de 64 bits tipo **long long**).

Clasificación y Conservación de Registros

- **Callee-saved (Preservados por el receptor - R4 a R11, y R13/SP):** Si la función secundaria necesita usar cualquiera de estos registros, debe guardar su valor en la pila al inicio y restaurarlo antes de retornar. El llamador asume que estos registros mantendrán sus valores intactos tras la llamada.
- **Caller-saved (Preservados por el emisor - R0 a R3, R12, y R14/LR):** Son volátiles. La función secundaria puede sobrescribirlos libremente sin salvarlos. Si el llamador

necesita conservar los datos que tenía en ellos, debe apilarlos *antes* de ejecutar la instrucción **BL**.

2. Declaración e Integración de Código

Para enlazar rutinas de C y ensamblador, se emplean directivas de visibilidad global:

Desde el Fichero C (**main.c**)

Declaramos la función ensamblador externa mediante el modificador **extern**:

```
#include <stdio.h>

// Declaración de función externa implementada en ensamblador
extern int sumar_tres_enteros(int a, int b, int c);

int main() {
    int resultado = sumar_tres_enteros(10, 20, 30);
    printf("Resultado: %d\\n", resultado);
    return 0;
}
```

Desde el Fichero Ensamblador (**sumar.s**)

Publicamos el símbolo del punto de entrada usando la directiva **.global**:

```
.text
.align 2
.global sumar_tres_enteros

sumar_tres_enteros:
    ; Parámetros de entrada en R0 (a), R1 (b) y R2 (c) según AAPCS
    ADD R0, R0, R1      ; R0 = a + b
    ADD R0, R0, R2      ; R0 = (a + b) + c (El resultado queda en R0 para el
retorno)
    BX LR               ; Retornar al llamador C
```

3. El Toque Informático

Auditor Automático de Cumplimiento AAPCS

El siguiente script en Python actúa como un linter de código estático básico que comprueba si un código ensamblador ARMv7 de muestra cumple con las normas de preservación de registros de la EABI/AAPCS al llamar a una subrutina:

```

def auditar_registro_aapcs(instrucciones_funcion):
    registros_modificados = set()
    registros_guardados = set()
    errores = []

    # Registros que obligatoriamente deben conservarse (callee-saved)
    callee_saved = {"R4", "R5", "R6", "R7", "R8", "R9", "R10", "R11"}

    for inst in instrucciones_funcion:
        partes = inst.replace(" ", "").replace("{", "").replace("}", "").split()
        if not partes:
            continue
        op = partes[0]

        if op == "PUSH":
            # Registrar qué registros se guardaron
            for reg in partes[1:]:
                registros_guardados.add(reg)
        elif op == "POP":
            pass # En epílogo se asume restauración
        else:
            # Identificar registros destino (primer argumento de la instrucción)
            if len(partes) > 1 and partes[1].startswith("R"):
                reg_destino = partes[1]
                registros_modificados.add(reg_destino)

    # Verificar violación de preservación
    for reg in registros_modificados:
        if reg in callee_saved and reg not in registros_guardados:
            errores.append(f"[ERROR] El registro callee-saved '{reg}' fue modificado pero no se guardó en PUSH.")

    print("=== INFORME DE AUDITORÍA AAPCS ===")
    if not errores:
        print(" - [OK] La subrutina cumple con la convención de preservación de registros.")
    else:
        for err in errores:
            print(f" - {err}")
    print("=====")

    # Prueba con subrutina que modifica R4 sin apilarlo
    codigo_incorrecto = [
        "PUSH {R5, LR}",
        "MOV R0, #10",
        "ADD R4, R0, R1", # R4 modificado pero no apilado
        "POP {R5, PC}"
    ]
    auditar_registro_aapcs(codigo_incorrecto)

```

4. Ejercicios Resueltos

Ejercicio 1

Una función en C llama a una rutina en ensamblador con la siguiente signatura: `extern int procesar_datos(int x1, int x2, int x3, int x4, int x5);` Indica detalladamente en qué registros y posiciones de memoria se sitúa cada uno de los parámetros recibidos por la función ensamblador al iniciarse.

Solución: Siguiendo las especificaciones de la convención de llamadas AAPCS de ARM:

- `x1`: Registro `R0`.
- `x2`: Registro `R1`.
- `x3`: Registro `R2`.
- `x4`: Registro `R3`.
- `x5`: Es el 5º parámetro (excedente). Se recibe en la cima de la pila, es decir, direccionado en la dirección apuntada por el Stack Pointer: `[SP]`.

Ejercicio 2

Explicar por qué es necesario usar la directiva `.global` en el código ensamblador y qué error se producirá en la fase de construcción del software si se omite.

Solución: La directiva `.global` le indica al ensamblador que exponga el símbolo de la función en la tabla de símbolos pública del archivo objeto (`.o`). Esto hace que el símbolo sea visible para otros módulos externos. Si se omite `.global`, el compilador de C compilará el archivo `main.c` asumiendo que la función existe, pero en la fase de **enlace (Linking)**, el Linker no podrá encontrar la dirección de memoria de la función en la tabla de símbolos del archivo ensamblador, provocando un error de enlace del tipo: `"Undefined reference to 'sumar_tres_enteros'".`

5. Ejercicios Propuestos

1. Dada la función en C: `int calcular(int a, int b);`. Si la implementamos en ensamblador, ¿estamos autorizados a modificar los registros `R0`, `R1`, `R2` y `R3` sin guardarlos en la pila? Justifica según la clasificación caller-saved/callee-saved.
2. Escribe una rutina en ensamblador ARM que sea llamada desde C para retornar el valor absoluto de un entero pasado como único parámetro (`extern int valor_absoluto(int num);`).

3. Investiga el papel del registro **R12 (IP)** en el estándar AAPCS. ¿Qué significa que sea un registro *intra-procedure-call scratch register* y en qué casos lo modifica el enlazador dinámico del sistema?

Tema 11: El Subsistema de Entrada/Salida: Registros del Controlador

Un computador no solo procesa datos internamente, también debe interactuar con el entorno exterior mediante periféricos (teclados, pantallas, sensores, tarjetas de red). El subsistema de Entrada/Salida (E/S) engloba la circuitería y protocolos necesarios para transferir información entre el procesador, la memoria y los controladores de dispositivos.

1. Métodos de Direccionamiento de Entrada/Salida

Para comunicarse con un periférico, la CPU debe poder acceder a los registros internos de su placa controladora. Existen dos enfoques de diseño:

E/S Mapeada en Puertos (Isolated I/O)

- **Principio:** El procesador dispone de un espacio de direcciones físicamente independiente para periféricos y utiliza instrucciones máquina específicas para acceder a él (ej. instrucciones **IN** y **OUT** en la arquitectura x86).
- **Limitación:** Requiere buses de control adicionales y restringe el uso de las instrucciones de procesamiento de datos en periféricos.

E/S Mapeada en Memoria (Memory-Mapped I/O - MMIO)

- **Principio (Estándar en ARM):** El controlador del periférico ocupa un rango de direcciones del espacio de direccionamiento general del sistema. La CPU se comunica con el dispositivo utilizando exactamente las mismas instrucciones de carga y almacenamiento empleadas con la memoria RAM (**LDR** y **STR**).
 - **Ventaja:** Es una arquitectura más limpia y permite aplicar cualquier modo de direccionamiento u operación de datos directamente sobre los registros del dispositivo.
-

2. Los Registros de la Placa Controladora de Dispositivo

El controlador de un periférico actúa como traductor entre los buses síncronos de alta velocidad del procesador y las señales eléctricas físicas del hardware exterior. Todo controlador expone de cara a la CPU una estructura lógica compuesta por tres tipos de registros de 8, 16 o 32 bits:

1. Registro de Datos (Data Register):

- *Función*: Almacena temporalmente la información enviada desde la CPU para ser transmitida al exterior (registro de salida), o la información recibida desde el exterior para ser leída por el procesador (registro de entrada).

2. Registro de Estado (Status Register):

- *Función (Solo Lectura para la CPU)*: Contiene banderas (*flags*) que reflejan el estado operativo actual del dispositivo. Ejemplos comunes:
 - **Ready** (Listo para transmitir/recibir).
 - **Busy** (Ocupado procesando).
 - **Error** (Fallo en la comunicación física).

3. Registro de Control (Control Register):

- *Función (Escritura/Lectura para la CPU)*: Contiene bits de configuración escritos por el procesador para gobernar el comportamiento del periférico. Ejemplos comunes:
 - Habilitar interrupciones del dispositivo.
 - Configurar velocidad de transmisión (Baudrate).
 - Iniciar o abortar una transferencia.

3. El Toque Informático

Simulador de Hardware de un Controlador de Periférico (UART)

El siguiente script en Python simula el comportamiento lógico de una placa controladora UART (puerto serie) mapeada en memoria. Muestra cómo al escribir y leer registros simulados mediante funciones equivalentes a **LDR** y **STR**, se altera el estado interno del hardware periférico:

```

class UARTController:
    def __init__(self):
        # Direcciones físicas simuladas de los registros del controlador (MMIO)
        self.ADDR_DATA = 0x40001000
        self.ADDR_STATUS = 0x40001004
        self.ADDR_CONTROL = 0x40001008

        # Estado interno de los registros (32 bits)
        self.reg_data = 0
        self.reg_status = 0x00000001 # Bit 0 (READY) inicializado en 1
        self.reg_control = 0

        # Búfer de transmisión físico
        self.buffer_tx = []

    def leer_registro(self, addr):
        if addr == self.ADDR_DATA:
            return self.reg_data
        elif addr == self.ADDR_STATUS:
            return self.reg_status
        elif addr == self.ADDR_CONTROL:
            return self.reg_control
        return 0

    def escribir_registro(self, addr, valor):
        if addr == self.ADDR_DATA:
            # Escribir en datos inicia la transmisión física
            self.reg_data = valor
            self.reg_status &= ~0x01 # Poner READY = 0 (Ocupado transmitiendo)
            self._simular_transmision_fisica(valor)
        elif addr == self.ADDR_CONTROL:
            self.reg_control = valor
            print(f"UART Control actualizado a: 0x{valor:08X}")

    def _simular_transmision_fisica(self, caracter_ascii):
        # El hardware envía los bits físicamente
        char = chr(caracter_ascii)
        self.buffer_tx.append(char)
        print(f"[HARDWARE UART]: Transmitiendo físicamente carácter '{char}'...")
        # Transmisión terminada: Restaurar READY = 1
        self.reg_status |= 0x01

# Simulación de la CPU escribiendo en la UART
uart = UARTController()
# 1. Comprobar si está lista leyendo el estado (READY bit)
estado = uart.leer_registro(uart.ADDR_STATUS)
if estado & 0x01:
    print("La UART está lista para transmitir.")
    # 2. Escribir el carácter ASCII 'H' (72) en el registro de datos (equivale a un STR
de la CPU)
    uart.escribir_registro(uart.ADDR_DATA, 72)

```

4. Ejercicios Resueltos

Ejercicio 1

Un periférico tiene sus registros de control, estado y datos mapeados en las siguientes direcciones de memoria física:

- **STATUS_REG**: `0x40002000` (Bit 0: **READY** a 1; Bit 1: **ERROR** a 1).
- **DATA_REG**: `0x40002004`. Escribir una rutina en ensamblador ARM para comprobar si el bit de error del periférico está activo y, en caso afirmativo, saltar a una etiqueta de control de fallos `gestionar_error`.

Solución:

```
LDR R0, =0x40002000 ; Cargar dirección de STATUS_REG en R0
LDR R1, [R0]         ; R1 = contenido de STATUS_REG
TST R1, #0x02        ; Comprobar bit 1 (ERROR) mediante AND lógica
BNE gestionar_error  ; Si el resultado no es cero (bit de error activo), saltar
```

Ejercicio 2

Explicar por qué es recomendable declarar como **volatile** en lenguaje C los punteros destinados a direccionar registros de Entrada/Salida mapeados en memoria (MMIO).

Solución: El compilador de C realiza optimizaciones agresivas. Si observa un bucle de lectura como:

```
while (*status_ptr == 0); // Esperar a que cambie el bit
```

El optimizador asumirá que, como el código del programa no modifica el valor apuntado por `status_ptr` dentro del bucle, el valor es constante. Por tanto, cargará el dato una sola vez en un registro de la CPU y entrará en un bucle infinito de CPU leyendo el registro interno, ignorando la memoria. Al declarar el puntero como **volatile**: `volatile int *status_ptr = (int*) 0x40002000;` Le indicamos al compilador que la dirección apuntada puede ser modificada por un hardware externo ajeno al software. Esto obliga a la CPU a realizar un acceso físico real a la memoria RAM/bus en cada iteración, permitiendo leer el estado del periférico actualizado en tiempo real.

5. Ejercicios Propuestos

1. Distingue entre el direccionamiento de Entrada/Salida **Mapeada en Memoria (MMIO)** y **Mapeada en Puertos (Isolated I/O)** en términos de espacio de direccionamiento e instrucciones máquina requeridas.
2. Un sensor de temperatura digital expone un registro de datos de 16 bits en la dirección **0x4000F000**. Escribe la instrucción de ensamblador ARM para cargar dicha lectura de datos sabiendo que los buses de datos de ARM son de 32 bits.
3. Investiga el papel del bit **READY** y del bit **BUSY** en un periférico. ¿Por qué es fundamental que la Unidad de Control del dispositivo coordine estos estados de forma independiente al procesador central?

Tema 12: Métodos de Transferencia: Polling o Consulta de Estado

Una vez que el procesador conoce la estructura de registros de un periférico, debe aplicar un método de sincronización para transferir datos de forma ordenada. El método más simple y directo es la **consulta de estado o espera activa (Polling)**.

1. El Algoritmo de Polling (Espera Activa)

En la Entrada/Salida por Polling, el procesador asume todo el control de la sincronización. La CPU ejecuta de forma continuada un bucle cerrado de lectura del registro de estado del controlador del dispositivo para comprobar si este se encuentra listo para transferir un dato.

Flujo de Operación para Transmisión

1. **Lectura del Estado:** La CPU lee la dirección de memoria del registro de estado (**STATUS_REG**).
2. **Máscara y Comprobación:** Aísla el bit **READY** (o el equivalente de transmisión vacía **TxEmpty**) mediante una operación lógica **AND**.
3. **Bucle de Espera Activa (Busy Waiting):**
 - Si **READY = 0**: El periférico está ocupado. La CPU salta de vuelta al paso 1, entrando en un bucle de espera continua.
 - Si **READY = 1**: El periférico está listo para recibir un nuevo dato.
4. **Escritura del Dato:** La CPU escribe el dato correspondiente en el registro de datos (**DATA_REG**), lo que inicia la transferencia física y hace que el controlador ponga el bit **READY = 0**.

Esquema de código en Ensamblador ARM

Asumiendo: **R0** = Dirección de **STATUS_REG**, **R1** = Dirección de **DATA_REG**, **R2** = Dato a transmitir.

```
espera_activa:
    LDR R3, [R0]          ; 1. Leer STATUS_REG
    TST R3, #0x01         ; 2. Comprobar bit 0 (READY)
```

```
BEQ espera_activa ; 3. Si READY == 0, seguir esperando en bucle
STR R2, [R1]      ; 4. Si READY == 1, escribir dato en DATA_REG
```

2. Análisis de Rendimiento y Sobrecarga de la CPU

Aunque la implementación por Polling es conceptualmente sencilla y requiere muy poca complejidad de hardware, presenta serias deficiencias de eficiencia en sistemas multitarea:

Ventajas

- **Mínima latencia de respuesta:** Como el procesador está monitorizando continuamente el periférico, detecta el cambio de estado de forma casi instantánea y transfiere el dato sin apenas retardo.
- **Sencillez de diseño:** No requiere circuitería especial de interrupción en la placa base ni controladores de interrupciones complejos.

Desventajas

- **Inutilización total de la CPU:** Mientras espera que un periférico lento (como un teclado o un puerto de red) esté listo, la CPU consume el 100% de sus ciclos de reloj ejecutando un bucle infructuoso, impidiendo realizar cualquier otro procesamiento útil de fondo.
- **Ineficiencia a escala:** Si el sistema tiene múltiples periféricos, realizar Polling a cada uno de ellos (round-robin polling) añade latencia y dificulta cumplir los requisitos de tiempo real de los dispositivos.

3. El Toque Informático

Simulación de un Bucle de Polling para Envío de Cadenas

El siguiente script en Python simula el bucle completo de espera activa de la CPU mientras transmite una cadena de texto a través del controlador de un puerto serie (UART). Muestra cómo se consume tiempo de procesamiento en espera de que el hardware esté disponible:

```
import time

class UARTPeriferico:
    def __init__(self):
        self.ready = True

    def transmitir_byte(self, char):
        # Simular retardo físico de transmisión por hardware
        self.ready = False
        print(f"\n[Hardware]: Iniciando transmisión de '{char}'...")
        time.sleep(0.1) # Retardo físico
        self.ready = True
        print(f"[Hardware]: Transmisión completa de '{char}'. UART lista.")

# Simulación de la CPU haciendo Polling
uart_dispositivo = UARTPeriferico()
mensaje = "ARM"

print("Iniciando transmisión de cadena mediante Polling...")
for caracter in mensaje:
    ciclos_de_espera = 0
    # Bucle de Espera Activa (Polling)
    while not uart_dispositivo.ready:
        ciclos_de_espera += 1
        # La CPU está bloqueada aquí consumiendo ciclos
        pass

    print(f"[CPU]: UART lista detectada (esperó {ciclos_de_espera} ciclos de software).")
    # Enviar el dato
    uart_dispositivo.transmitir_byte(caracter)
```

4. Ejercicios Resueltos

Ejercicio 1

Diseñar una subrutina en ensamblador ARM llamada `enviar_cadena` que transmita una secuencia de caracteres (cadena de caracteres terminada en cero, formato ASCIIIZ) cuya dirección de memoria inicial está en `R2`, utilizando Polling. Las direcciones de los registros son:

- `STATUS_REG = 0x40003000` (Bit 0: `READY`).
- `DATA_REG = 0x40003004`.

Solución:

```
.text
.global enviar_cadena

enviar_cadena:
    PUSH {R4, R5, LR}    ; Guardar contexto y retorno
    LDR R4, =0x40003000   ; R4 = Dirección de STATUS_REG
    LDR R5, =0x40003004   ; R5 = Dirección de DATA_REG

bucle_caracter:
    LDRB R0, [R2], #1     ; Cargar carácter actual y avanzar puntero de cadena
    CMP R0, #0            ; Comprobar si es el fin de la cadena (carácter nulo)
    BEQ fin_transmision   ; Si R0 == 0, terminar

esperar_uart:
    LDR R3, [R4]          ; Leer STATUS_REG
    TST R3, #0x01         ; Comprobar bit 0 (READY)
    BEQ esperar_uart      ; Si es 0 (ocupado), seguir haciendo polling

    STRB R0, [R5]         ; Si es 1 (listo), escribir carácter en DATA_REG
    B bucle_caracter      ; Procesar siguiente carácter

fin_transmision:
    POP {R4, R5, PC}      ; Restaurar registros y retornar
```

5. Ejercicios Propuestos

1. Dada una tasa de transmisión de red de 10.000 bytes por segundo. Si la CPU ejecuta un bucle de Polling donde cada lectura de estado tarda $1\ \mu s$, calcula cuántos ciclos de consulta estériles realiza la CPU en promedio entre la llegada de cada byte de datos.
2. Propón una alternativa de diseño de hardware o software para evitar que la CPU quede bloqueada en un bucle de espera activa mientras aguarda a que un usuario pulse una tecla (dispositivo extremadamente lento).
3. Explica qué es el efecto de *Live-lock* en E/S por consulta de estado y en qué condiciones puede llegar a congelar por completo la ejecución de procesos prioritarios del sistema operativo.

Tema 13: Mecanismo de Interrupciones y Excepciones en ARM

Para resolver las ineficiencias de la Entrada/Salida por Polling, la arquitectura de computadores introduce el mecanismo de **Interrupciones**. Este sistema permite a los periféricos notificar de forma asíncrona al procesador cuando ocurre un evento relevante, permitiendo a la CPU dedicarse a otras tareas útiles mientras el dispositivo trabaja de forma autónoma.

1. Conceptos y Diferencia entre Excepciones e Interrupciones

Aunque a nivel de hardware se gestionan de forma muy similar, conceptualmente existe una distinción clara:

- **Excepciones (Síncronas)**: Son provocadas de forma interna por la propia ejecución de las instrucciones del programa. Ocurren en instantes de tiempo predecibles.
Ejemplos:
 - División por cero.
 - Fallo de página de memoria.
 - Instrucción no definida.
 - Llamadas al sistema operativo (**SVC** - *Supervisor Call*).
 - **Interrupciones (Asíncronas)**: Son provocadas de forma externa por eventos físicos de hardware ajenos a la secuencia de instrucciones que corre la CPU. Ocurren de forma impredecible en cualquier instante de tiempo. Ejemplos:
 - Llegada de un paquete de datos por la tarjeta de red.
 - Pulsación de una tecla por parte del usuario.
 - Vencimiento del temporizador de hardware (*Timer ticks*).
-

2. El Proceso de Atención a una Interrupción (Hardware/Software)

Cuando un periférico activa la línea física de interrupción (**IRQ** o **FIQ** en ARM), el procesador y el sistema operativo coordinan el siguiente flujo de acciones:

1. Fase de Hardware (CPU)

1. La CPU finaliza la ejecución de la instrucción máquina actual.
2. Salva el registro de estado actual copiándolo al **SPSR** (*Saved Program Status Register*) del modo correspondiente.
3. Guarda el valor de retorno en el registro de enlace especial **LR_irq** (guardando $PC + 4$).
4. Cambia automáticamente el modo del procesador a **IRQ Mode** y deshabilita nuevas interrupciones (pone el bit **I = 1** en el CPSR).
5. Actualiza el contador de programa **PC** con la dirección de memoria reservada correspondiente de la **Tabla de Vectores de Interrupción (IVT)**.

2. Fase de Software (Sistema Operativo e ISR)

6. La posición de la IVT contiene una instrucción de salto directo a la **Rutina de Servicio a la Interrupción (ISR)** de ese periférico.
7. **Prólogo de la ISR**: Salva en la pila todos los registros de trabajo de la CPU que va a utilizar para no destruir el contexto del programa interrumpido.
8. **Cuerpo de la ISR**: Procesa la información del periférico y escribe en la controladora para indicarle que el dato ya se leyó (borrar el bit de petición).
9. **Epílogo de la ISR**: Restaura los registros guardados de la pila.
10. **Retorno de la ISR**: Ejecuta una instrucción de retorno dedicada que restaura simultáneamente el **PC** y el registro de estado **CPSR** desde el **SPSR**: **SUBS PC, LR, #4** (en ARM, debido al desfase del pipeline).

3. La Tabla de Vectores de Interrupción (IVT) en ARM

ARM reserva las posiciones más bajas de la memoria física para la Tabla de Vectores de Interrupción. Cada entrada de la tabla tiene un tamaño fijo de 4 bytes, suficiente para albergar una instrucción de salto (**LDR PC, [PC, #offset]**):

Dirección Vector	Excepción / Interrupción	Descripción
0x00000000	Reset	Arranque o reinicio físico del procesador.
0x00000004	Undefined Instruction	La CPU intenta ejecutar un código de operación inválido.
0x00000008	Software Interrupt (SVC)	Llamada al sistema para invocar servicios del S.O.
0x0000000C	Prefetch Abort	Fallo al intentar buscar una instrucción en memoria.
0x00000010	Data Abort	Fallo al intentar leer/escribir datos en memoria (ej: alineación).
0x00000018	IRQ	Petición de interrupción externa normal.
0x0000001C	FIQ	Petición de interrupción externa rápida (Fast IRQ).

4. El Toque Informático

Simulador del Control de Flujo de Interrupciones

El siguiente script en Python simula el comportamiento de una CPU ejecutando un bucle principal de procesamiento, que es interrumpido de forma asíncrona por un temporizador de hardware. Muestra cómo se guarda el contexto y se bifurca a la tabla de vectores y a la ISR:

```
class CPUSimulator:
    def __init__(self):
        self.pc = 0x8000 # Programa principal
        self.registers = {"R0": 42, "R1": 99}
        self.cpsr = "MODO_USUARIO"
        self.spsr = ""
        self.lr_irq = 0

    def bucle_principal(self):
        print(f"[CPU]: Ejecutando código principal. PC = 0x{self.pc:04X} | R0 = {self.registers['R0']}")
        self.pc += 4

    def simular_peticion_irq(self):
        print("\n!!! [HARDWARE]: Petición IRQ (Timer Tick) activa !!!")
```

```

# 1. Salvar contexto de hardware
self.spsr = self.cpsr
self.lr_irq = self.pc + 4 # Dirección de retorno
self.cpsr = "MODO_IRQ"

# 2. Saltar al Vector de IRQ en la IVT (Dirección 0x18)
self.pc = 0x0018
print(f"[CPU]: Modo cambiado a {self.cpsr}. LR_irq = 0x{self.lr_irq:04X}. PC saltando a IVT [0x{self.pc:02X}].")

# 3. Simular la ejecución de la ISR
self._ejecutar_isr()

def _ejecutar_isr(self):
    print(f"[IVT 0x0018]: Redirigiendo a ISR (LDR PC, =0x9000)")
    self.pc = 0x9000
    print(f"[ISR 0x{self.pc:04X}]: Guardando R0 y R1 en la pila.")
    # Procesamiento simulado
    print("[ISR]: Procesando tick del reloj del sistema...")
    print(f"[ISR 0x{self.pc:04X}]: Restaurando R0 y R1. Retornando.")

    # Retorno: Restaurar PC y CPSR (SUBS PC, LR, #4)
    self.pc = self.lr_irq - 4
    self.cpsr = self.spsr
    print(f"[CPU]: Retornado con éxito al programa principal. PC = 0x{self.pc:04X} | Modo = {self.cpsr}\n")

# Ejecución
cpu = CPUSimulator()
cpu.bucle_principal()
cpu.bucle_principal()
cpu.simular_peticion_irq()
cpu.bucle_principal()

```

5. Ejercicios Resueltos

Ejercicio 1

Explicar por qué al retornar de una rutina de servicio a la interrupción de tipo IRQ en un procesador ARM, se debe restar 4 al registro de enlace **LR** antes de copiarlo a **PC** (**SUBS PC, LR, #4**), en lugar de retornar directamente con **MOV PC, LR**.

Solución: ARM utiliza una arquitectura con **cauce segmentado (pipelined)** de 3 etapas (Búsqueda, Decodificación y Ejecución). Debido a esto:

- El Program Counter **PC** siempre apunta 8 bytes (dos instrucciones) por delante de la instrucción que se está ejecutando físicamente.

- Cuando se activa la señal física de interrupción **IRQ**, el procesador guarda en **LR_irq** la dirección de la instrucción que estaba decodificando en ese momento ($PC + 4$).
- Sin embargo, la instrucción que estaba en fase de decodificación no llegó a completarse (la CPU solo termina la que estaba en fase de ejecución).
- Por lo tanto, al retornar de la ISR, debemos volver a ejecutar esa instrucción no completada. Para ello, debemos retroceder 4 bytes respecto a la dirección guardada en **LR**:

$$\text{Dirección de Retorno Real} = LR - 4$$

La instrucción máquina **SUBS PC, LR, #4** resta 4 a **LR**, lo escribe en **PC** y restaura el estado del **CPSR** de forma atómica en un único ciclo de reloj.

Ejercicio 2

Identificar la diferencia entre las interrupciones **IRQ** y **FIQ** de la arquitectura ARM. ¿Qué optimizaciones físicas implementa **FIQ** para ser más rápida?

Solución:

- **IRQ (Interrupt Request)**: Es la línea de interrupción estándar de propósito general.
- **FIQ (Fast Interrupt Request)**: Es la línea de interrupción de alta velocidad para dispositivos críticos.
- **Optimizaciones de FIQ**:
 1. **Banco de registros duplicado (*Register Shadowing*)**: FIQ tiene sus propios registros físicos dedicados **R8_fiq** a **R14_fiq**. Esto permite que la ISR de FIQ use estos registros directamente sin necesidad de apilarlos en la pila de memoria (ahorrando múltiples ciclos de lectura/escritura en RAM).
 2. **Ubicación en la IVT**: El vector de FIQ está en la dirección **0x0000001C**, que es la última posición física de la IVT. Al no haber más vectores debajo, la rutina de servicio a la interrupción (ISR) de FIQ puede colocarse directamente a partir de la dirección **0x1C** sin necesidad de realizar una instrucción de salto intermedia.

6. Ejercicios Propuestos

1. Distingue de forma detallada entre una **excepción síncrona** y una **interrupción asíncrona**, aportando dos ejemplos reales de cada una que afecten a la ejecución de un sistema informático.

2. Un programador diseña una rutina de servicio a la interrupción (ISR) pero olvida guardar el registro **R0** en la pila al inicio. Describe las posibles consecuencias y fallos colaterales en el programa principal interrumpido.
3. Investiga el papel del **Controlador Vectorizado de Interrupciones (VIC o NVIC)** en la arquitectura ARM. ¿Cómo ayuda a gestionar las prioridades cuando ocurren múltiples interrupciones de hardware simultáneamente?

Glosario de Términos

- **AAPCS (Procedure Call Standard for the ARM Architecture):** Estándar que define el uso homogéneo de registros y pila para el paso de parámetros y retorno de datos entre funciones en ARM.
- **ALU (Arithmetic Logic Unit):** Unidad funcional del procesador que ejecuta operaciones aritméticas y lógicas básicas.
- **Barrel Shifter:** Dispositivo hardware de ARM que desplaza o rota operandos de un registro de forma simultánea a la ejecución de una instrucción.
- **Bloque de Activación (Stack Frame):** Espacio reservado dinámicamente en la pila para variables locales, parámetros y contexto de una invocación a una subrutina.
- **CPSR (Current Program Status Register):** Registro que contiene las banderas de condición aritméticas (N, Z, C, V) y el modo de control de la CPU.
- **Excepción:** Evento síncrono interno desencadenado por la ejecución de una instrucción que desvía el control del programa.
- **FIQ (Fast Interrupt Request):** Línea de interrupción asíncrona rápida de ARM optimizada con bancos de registros duplicados dedicados.
- **Frame Pointer (FP):** Registro R11 que provee una dirección base estable para indexar variables locales y parámetros excedentes dentro de un marco de pila.
- **Interrupción:** Señal asíncrona de hardware generada por un periférico para notificar a la CPU la existencia de un evento de Entrada/Salida.
- **ISR (Interrupt Service Routine):** Rutina software encargada de atender la petición específica de un dispositivo de hardware y liberar su línea de interrupción.
- **IVT (Interrupt Vector Table):** Tabla reservada en memoria física que asocia cada tipo de excepción o interrupción con su correspondiente vector de salto a la ISR.
- **Link Register (LR):** Registro R14 que almacena automáticamente la dirección de retorno al saltar a una subrutina mediante la instrucción **BL**.
- **MMIO (Memory-Mapped I/O):** Método de Entrada/Salida que mapea los registros del controlador del periférico en el espacio general de memoria RAM, accesibles mediante **LDR** y **STR**.
- **Polling:** Entrada/Salida síncrona en la que la CPU consulta indefinidamente en un bucle cerrado (espera activa) el estado de un controlador de dispositivo.
- **Program Counter (PC):** Registro R15 que contiene la dirección de memoria de la instrucción máquina actualmente en fase de ejecución por la CPU.
- **SPSR (Saved Program Status Register):** Registro auxiliar donde la CPU preserva de forma automática el CPSR previo antes de conmutar a un modo de excepción.
- **Stack Pointer (SP):** Registro R13 que guarda la dirección de memoria de la última posición de datos ocupada de la pila en ejecución.

Bibliografía Recomendada

1. **Patterson, D. A., & Hennessy, J. L. (2020).** *Computer Organization and Design ARM Edition: The Hardware Software Interface*. Morgan Kaufmann.
 - *Nota:* La referencia académica internacional más importante y detallada sobre la interfaz hardware/software del repertorio ARM.
2. **Yiu, J. (2013).** *The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors* (3rd ed.). Newnes.
 - *Nota:* Una obra técnica práctica muy rigurosa orientada al estudio del banco de registros, excepciones y controladores en microcontroladores ARM.
3. **Furber, S. (2000).** *ARM System-on-Chip Architecture* (2nd ed.). Addison-Wesley.
 - *Nota:* Escrito por uno de los diseñadores originales de la arquitectura ARM, proporciona una visión profunda y detallada sobre la ruta de datos y la organización física.
4. **Harris, D. M., & Harris, S. L. (2015).** *Digital Design and Computer Architecture: ARM Edition*. Morgan Kaufmann.
 - *Nota:* Un texto didáctico excelente que vincula compuertas lógicas y diseño digital con la programación en ensamblador y microarquitectura ARM.